UNITED STATES PATENT AND TRADEMARK OFFICE

BEFORE THE PATENT TRIAL AND APPEAL BOARD

ServiceNow, Inc.
Petitioner

v.

Hewlett-Packard Company
Patent Owner

U.S. Patent No. 7,392,300
Filing Date: January 8, 2004
Issue Date: June 24, 2008

TITLE: METHOD AND SYSTEM FOR MODELLING A COMMUNICATIONS NETWORK

DECLARATION OF TAL LAVIAN, Ph.D.

Inter Partes Review No. 2015-__

Table of Contents

Page

I. BRIEF SUMMARY OF MY OPINIONS 1

II. INTRODUCTION AND QUALIFICATIONS 2

A. Qualifications and Experience 2

B. Materials Considered 7

III. PERSON OF ORDINARY SKILL IN THE ART 8

IV. RELEVANT TECHNOLOGY BACKGROUND 9

A. Computer Networks and Network Management Systems 10

B. Structured Data and the Extensible Markup Language (XML) 15

V. THE PATENT'S TECHNIQUE FOR MODELING COMMUNICATION NETWORKS 19

A. The Specification 19

B. The Claims of the '300 Patent 22

C. Claim Construction 23

1. "Network Representation" 24

2. "Network Model" 26

3. "Network Event" 26

VI. APPLICATION OF THE PRIOR ART TO THE CLAIMS OF THE '300 PATENT 28

A. Claims 1, 7, 8, 10, 21 and 22 Are Obvious Over Matheny in view of XML in a Nutshell, Hamner and Arrouye 34

1. Each Limitation of Claim 1 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 34

(a) "generating a network representation using computer-readable code, the computer-readable code representing structured information" 34

(b) "parsing the network representation" 38

Table of Contents
(continued)

Page

(c) "generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects" 43

(d) "storing the network model in memory" 46

(e) "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects…" 47

(f) "…and the processing further includes determining an order of operation on the one or more network objects" 57

2. Each Limitation of Claim 7 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 64

3. Each Limitation of Claim 8 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 64

4. Each Limitation of Claim 10 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 66

5. Each Limitation of Claim 21 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 67

6. Each Limitation of Claim 22 is Disclosed by Matheny, XML in a Nutshell, Hamner and Arrouye 70

B. Claims 1, 7, 8, 10, 21 and 22 Are Obvious Over Matheny in view of XML in a Nutshell, Hamner and Pitt 71

VII. CONCLUSION 77

I, Tal Lavian, Ph.D., declare as follows:

1. I have personal knowledge of the facts stated in this declaration, and could and would testify to these facts under oath if called upon to do so.

2. I have been retained by counsel for ServiceNow, Inc. (Petitioner) in this case as an expert in the relevant art. I have been asked to provide my opinions relating to claims 1, 7, 8, 10, 21 and 22 of U.S. Patent No. 7,392,300 by Raghu Ananthrangachar et al. ("'300 patent"), which I understand is assigned to Hewlett-Packard Company.

I. BRIEF SUMMARY OF MY OPINIONS

3. Claims 1, 7, 8, 10, 21 and 22 purport to disclose a method, system and computer program for modelling a computer network. The alleged invention of these claims is, at a high level, straightforward and easy to understand. The claims require generation of a "network representation" that describes the devices in a network. This "network representation" must comprise eXtensible Markup Language (XML) or some other form of "structured information." This network representation, once generated, is parsed and then used to generate a "network model," which in turn is used to process network events.

4. Claims 1, 7, 8, 10, 21 and 22 do not describe anything that was new or non-obvious by the time the application for the ’300 patent was filed in January 2004. The generation and use of a “network representation” in XML – the feature to which the majority of the specification is devoted – was disclosed by Matheny. Matheny not only discloses the claimed “network representation,” but does so using language remarkably similar to the challenged claims. The remaining limitations in the challenged claims, including the creation of a “network model” and using it to process network events, recite basic functionality found of network management systems that predate the patent by nearly a decade, as shown by the prior art references below including Hamner.

5. Because each element of the challenged claims is disclosed or suggested by the prior art I describe in my Declaration, and a person of ordinary skill in the art would have had ample motivation to combine, each challenged claim is obvious. The bases for my opinions are set forth in detail below.

II. INTRODUCTION AND QUALIFICATIONS

A. Qualifications and Experience

6. I possess the knowledge, skills, experience, training and the education to form an expert opinion and testimony in this case. A detailed record

of my professional qualifications, including a list of patents and academic and professional publications, is set forth in my curriculum vitae attached to this declaration as **Exhibit A**.

7. I have more than 25 years of experience in the networking, telecommunications, Internet, and software fields. I received a Ph.D. in Computer Science from the University of California at Berkeley in 2006 and obtained a Master's of Science ("M.Sc.") degree in Electrical Engineering from Tel Aviv University, Israel, in 1996. In 1987, I obtained a Bachelor of Science ("B.Sc.") in Mathematics and Computer Science, also from Tel Aviv University.

8. I am currently employed by the University of California at Berkeley and was appointed as a lecturer and Industry Fellow in the Center of Entrepreneurship and Technology ("CET") as part of UC Berkeley College of Engineering. I have been with the University of California at Berkeley since 2000 where I served as Berkeley Industry Fellow, Lecturer, Visiting Scientist, Ph.D. Candidate, and Nortel's Scientist Liaison, where some positions and projects were done concurrently, others sequentially.

9. I have more than 25 years of experience as a scientist, educator and technologist, and much of my experience relates to computer networking

technologies. For eleven years from 1996 to 2007, I worked for Bay Networks and Nortel Networks. Bay Networks was in the business of making and selling computer network hardware and software. Nortel Networks acquired Bay Networks in 1998, and I continued to work at Nortel after the acquisition. Throughout my tenure at Bay and Nortel, I held positions including Principal Scientist, Principal Architect, Principal Engineer, Senior Software Engineer, and led the development and research involving a number of networking technologies.

10. Prior to that, from 1994 to 1995, I worked as a software engineer and team leader for Aptel Communications, designing and developing mobile wireless devices and network software products. From 1990 to 1993, I worked as a software engineer and team leader at Scitex Ltd., where I developed system and network communications tools (mostly in C and C++).

11. I have extensive experience in the area of network communications and Internet technologies including design and implementation of computer-based systems for managing communications networks, including the ability to monitor and provision networks. While with Nortel Networks and Bay Networks (mentioned above) my work involved the research and development of these technologies. For example, I wrote software for Bay Networks and Nortel

Networks Web based network management for Bay Networks switches. I developed Simple Network Management Protocol (SNMP) software for Bay Network switches and software interfaces for Bay Networks' Optivity Network Management System. I wrote software for Java based device management including software interface to the device management and network management for the Accelar routing switch family network management system.

12. I have extensive experience in network communications, including control and management of routing and switching architectures and protocols in layers 1-7 of the OSI model.. Much of my work for Nortel Networks (mentioned above) involved the research and development of network communications technologies. For example, I wrote software for Bay Networks and Nortel Networks switches and routers, developed network technologies for the Accelar 8600 family of switches and routers, the OPTera 3500 SONET switches, the OPTera 5000 DWDM family, and the Alteon L4-7 switching product family. In my lab, I installed, configured, managed and tested many network communications equipment of competitors such as Cisco Systems, Juniper Networks, Extreme Networks, Lucent and Alcatel.

13. I am named as a co-inventor on more than 80 issued patents and I co-authored more than 25 scientific publications, journal articles, and peer-reviewed papers. Furthermore, I am a Senior Member of the Institute of Electrical and Electronics Engineers ("IEEE").

14. I currently serve as a Principal Scientist at my company Telecomm Net Consulting Inc., where I develop network communication technologies and provide research and consulting in advanced technologies, mainly in computer networking and Internet technologies. In addition, I serve as a Co- Founder and Chief Technology Officer (CTO) of VisuMenu, Inc., where I design and develop architecture of visual IVR technologies for smartphones and wireless mobile devices in the area of network communications. The system is based on cloud networking and cloud computing utilizing Amazon Web Services.

15. Additional details of my background are set forth in my curriculum vitae, attached as **Exhibit A** to this Declaration, which provides a more complete description of my educational background and work experience. I am being compensated for the time I have spent on this matter at the rate of $400 per hour. My compensation does not depend in any way upon the outcome of this

proceeding. I hold no interest in the Petitioner (ServiceNow, Inc.) or the patent owner (Hewlett-Packard Company).

B. Materials Considered

16. The analysis that I provide in this Declaration is based on my education and experience in the field of computer networking technologies, as well as the documents I have considered, including U.S. Patent No. 7,392,300 B2 ("'300 patent") **[Ex. 1001]**, which states that it issued from an application filed on January 8, 2004. I also reviewed the prosecution history for the '300 patent.

17. I reviewed various documents dated prior to January 2004 describing the state of the art at the time of the alleged invention of the '300 patent. The prior art documents that I rely upon in this Declaration as disclosing the limitations of the claims are listed in the table below:

Exhibit No.	Description of Document
1003	U.S. Patent App. Pub. No. 2002/0161883 to David Matheny et al., published on October 31, 2002
1004	Elliotte Rusty Harold et al., *XML in a Nutshell* (2001), pp.xi-xvi, 3-25
1005	U.S. Patent No. 5,796,951 to John F. Hamner et al., which issued on August 18, 1998
1006	U.S. Patent No. 6,256,635 to Yan Arrouye et al., which issued on July 3, 2001

Exhibit No.	Description of Document
1007	U.S. Patent No. 5,717,934 to Ronald L. Pitt et al., which issued on February 10, 1998

This Declaration also cites the following additional prior art document for purposes of describing the relevant technology, including the relevant state of the art at the time of the alleged invention of the ’300 patent:

Exhibit No.	Description of Document
1008	Microsoft Computer Dictionary (3rd ed. 1997), Microsoft Press, pp.422-423

III. PERSON OF ORDINARY SKILL IN THE ART

18. I understand that an assessment of claims of the ’300 patent should be undertaken from the perspective of a person of ordinary skill in the art as of the earliest claimed priority date, which I understand is January 8, 2004.

19. In my opinion, a person of ordinary skill in the art as of January 2004 would possess, at a minimum, a bachelor’s degree in electrical engineering or computer science (or an equivalent degree or experience) with at least four years of practical experience (or a similar advanced degree with at least four years of practical experience) in the design and implementation of computer-based systems for managing communications networks, which should include systems

that provide the ability to monitor, model and provision networks. I expect that such a person would also have at least four years of practical experience with computer programming in this field, and a basic understanding of structured or semi-structured data formats such as the Extensible Markup Language (XML).

20. Although my qualifications and experience exceed those of the hypothetical person having ordinary skill in the art defined above, my analysis and opinions regarding the ’300 patent have been based on the perspective of a person of ordinary skill in the art as of January 8, 2004.

21. My opinions regarding the level of ordinary skill in the art are based on, among other things, my over 25 years of experience in the field of network communications, computer science and engineering, my understanding of the basic qualifications that would be relevant to an engineer or scientist tasked with investigating methods and systems in the relevant area, and my familiarity with the backgrounds of colleagues and co-workers, both past and present.

IV. RELEVANT TECHNOLOGY BACKGROUND

22. The ’300 patent, entitled “Method and System for Modelling a Communications Network,” discloses a technique for generating a model of a

computer network. ('300, Abstract, 1:39-48.) In this section, I will provide a brief overview of the technology pertinent to the '300 patent as of January 2004.

A. Computer Networks and Network Management Systems

23. Most computer users today, and in January 2004, are familiar with the basic idea behind a computer communications network. Generally speaking a computer network is a group of computing devices (often called "nodes") that are connected to each other. These nodes communicate with each other over some kind of communications medium, such as a wired or wireless connection.

24. Computer networks come in many different varieties. The term "**local area network**" or "LAN" typically refers to a network having nodes restricted to a limited area, such as an office building. On the other end of the spectrum, a "**wide area network**" or "WAN," as its name implies, typically includes nodes that are connected over a much larger area, such as a large geographic region or even across international borders. Many often regard the "Internet" as an example of a very large and worldwide wide area network.

25. A related type of network, known as a "**virtual private network**" or "VPN," is specifically mentioned in the Background of the '300 patent. ('300, 1:17-19.) Virtual private networks combine the features of local area networks

and wide area networks to provide network computing within a large enterprise, for example, to connect computers located in geographically dispersed offices of a large corporation. With a virtual private network, the computing devices on the network often communicate over distances using public or shared networks, such as the Internet. ('300, 1:17-19.) Virtual private networks typically rely on additional technologies, such as authentication and encryption, to prevent unauthorized access to the network and to prevent interception of data as it is transmitted across the public network.

26. But with any type of network, as the network grows and the number of devices connected to it increases, there is a corresponding increase in the need for software tools to monitor and manage the network and the computing devices connected to it. An entire industry, often referred to as "**network management**," grew out of this need and was firmly in place by the time the '300 patent was filed in January 2004. Network management tools allow network administrators and other persons to, among other things, identify and configure devices on the network, monitor their performance, and in some cases, create graphical displays to visualize the network devices and their interconnections.

27. Almost all network equipment vendors provided such network management tool to configure and manage their equipment; some vendors offered several management tools to manage different product lines. For example, Cisco Systems, Nortel Networks, and Juniper Networks offered such tools, and in addition, companies like IBM, HP, Tivoli and Computer Associates (CA) offered more sophisticated tools to manage networks of multiple vendors. Starting around 1996, I personally used several of these tools to configure, manage, and test networks in my lab at Bay Networks and Nortel Networks.

28. The core concepts of managing networks were well-known in the industry, and in fact, it would have been unlikely to configure, manage and operate networks without network management tools. Network operators, network service providers and Internet service providers used these types of tools in a large scale, and it could have been extremely hard to run their networks without these tools. The ideas contained therein were mainstream at the time of the invention.

29. As explained in the Background of the ’300 patent: “A variety of software products and applications currently exist to facilitate network development, management, and maintenance.” (’300, 1:28-30.)

30. One example of such a system is disclosed in U.S. Patent No. 5,796,951 to John F. Hamner ("**Hamner**") **[Ex. 1005]**, which issued from an application filed in December 1995, more than eight years before the filing of the application for the '300 patent. Hamner explains that "as users place more reliance upon computer networks," those users "also demand more sophisticated capabilities for managing these networks. Various different software packages exist for providing users with such capabilities." (Hamner, 1:20-24.) Hamner purports to provide such a system that provides a network "Management Console" shown in Figure 2A that lists the devices on the network and the tasks that can be performed on the devices:

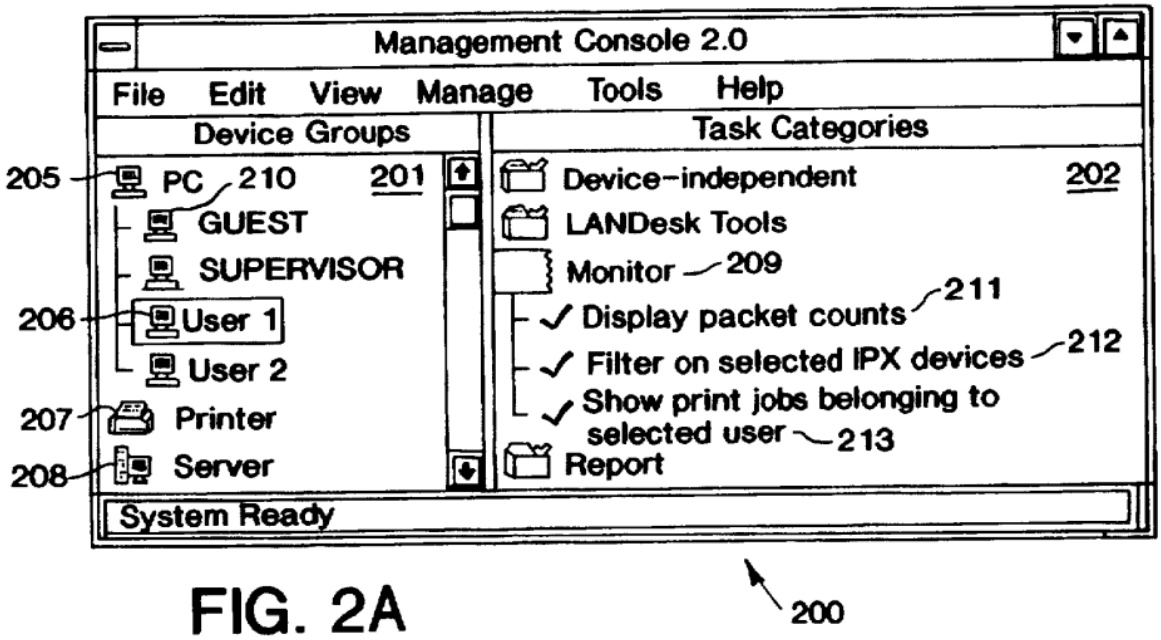


FIG. 2A

(Hamner, Fig. 2A.)

31. The exemplary system in Figure 2A of Hamner includes a device window (**201**) on the left that shows the various devices on the network organized based on the group in which they belong (*e.g.* the "PC" device group (**205**), the "Printer" device group (**207**), or "Server" device group (**208**)). (Hamner, 4:3-16.) Figure 2A shows the network modelled into a "tree" hierarchy allowing the user to click on a particular device group, such as the PC device group (**205**) in Figure 2A, to view all of the devices that fall within the selected group. (Hamner, 4:10-16.) The system in Hamner also contemplates "schematic, geographic or other views of the layout of the network, in addition to or instead of a tree hierarchy view." (Hamner, 4:53-56.)

32. The right side of Figure 2A includes a task window (**202**) that displays tasks and categories of network management tasks that can be performed on a network device. (Hamner, 4:18-32.) "The user can cause any displayed task to be performed upon a device." (Hamner, 4:33-34.) "Some examples of tasks which might be performed on various devices in a network are: viewing the screen of a particular PC; displaying packet counts; running a report; executing a remote virus

scan; rebooting selected workstations; displaying print jobs; or, displaying non-functioning printers." (Hamner, 3:51-56.)

B. Structured Data and the Extensible Markup Language (XML)

33. Another concept mentioned in the '300 patent is the use of "structured information," such as information represented using the Extensible Mark-up Language (XML). As I explain below, one of the steps of the alleged invention is to generate structured information (such as XML) to describe the objects in the network. ('300, 3:63-4:10, 9:43-45 (claim 1).)

34. Computer scientists often describe information as being either "**structured**" or "**unstructured**." Unstructured data generally refers to information that does not follow any specific format or organization. Examples of unstructured data include free form text and uncompressed photo image or audio data. Structured data, on the other hand, refers to data that follows a specific organizational model. One example of structured data is information stored in a relational database according to a specific schema, which specifies how the data is stored and how data elements relate to each other.

35. Another example of structured information described in the '300 patent is data that has been represented in the Extensible Markup Language or

"XML." XML generally refers to an industry-standard set of rules for encoding information. As explained in Elliotte Rusty Harold et al., *XML in a Nutshell* (2001) ("**XML in a Nutshell**") **[Ex. 1004]**, XML "defines a generic syntax used to mark up data with simple, human-readable tags." (XML in a Nutshell, at 3.)

36. XML "provides a standard format for computer documents," and "is flexible enough to be customized for domains as diverse as web sites, electronic data interchange, vector graphics, genealogy, real-estate listings, object serialization, remote procedure calls and voice-mail systems." (*Id.*) The following shows how information about two books could be represented using XML:

```
<book>
 <title> A Tale of Two Cities </title>
 <author> Charles Dickens </author>
 <year> 1859 </year>
</book>
<book>
 <title> Leaves of Grass </title>
 <author> Walt Whitman </author>
 <year> 1855 </year>
</book>
```

The XML in the above example includes *start tag*s and *end tags*. For example <year> is a start tag for the object "year" (value=1859), and </year> is an end tag

for the same object "year". A start tag includes the less-than character ("<"), followed by the name of the tag and with the greater-than character (">"). (XML in a Nutshell, at 12-13.) For example, the <book>, <title>, <author>, and <year> are start tags. Each tag defines an element in XML and may be followed by additional tags or contents. XML tags can be nested to create hierarchical relationships, as shown by the "<book>" tag that has child tags for "<title>," "<author>" and "<year>."

37. The example above also shows *end tags*, each end-tag corresponding to an earlier start-tag. An end tag includes a less-than and slash character ("</"), followed by the name of the tag and a greater-than character (">"). Accordingly, </book>, </title>, </author> and </year> in the above example are end-tags.[1]

38. XML is an industry standard for encoding information and is widely used for sharing information between computer systems and application

---

[1] The '300 patent refers to XML as an example of "structured" computer-readable information. ('300, Abstract, Claims 1 & 7.) Many persons of ordinary skill in the art would regard XML as accurately classified as "semi-structured data" because it follows a more relaxed set of rules than strictly structured data such as a database schema. Because the distinction between "structured" and "semi-structured data" is not material to the opinions set forth in this Declaration, I will refer to XML as an example of "structured information" consistent with the description in the '300 patent.

programs. (XML in a Nutshell, at 6.) As explained in *XML in a Nutshell*, "XML is one of the most important developments in document syntax in the history of computing," and had "become the syntax of choice for newly designed document formats across almost all computer applications." (*Id.* at Preface, xi.) "In many ways, XML is the most portable and flexible document format designed since the ASCII text file." (*Id*. at 6.)

V. THE PATENT'S TECHNIQUE FOR MODELING COMMUNICATION NETWORKS

A. The Specification

39. The '300 patent, entitled "Method and System for Modelling a Communications Network," generally describes a computer-based technique for creating a model of a communications network, the model identifying the objects in the network (e.g. devices) and the relationships between them. According to the specification, "[t]he communications network modeled may be any type of network such as, for example, a local area network, wide area network, virtual private network, or any other suitable network configuration." ('300, 2:37-41.)

40. The process of modeling a communications network is summarized in Figure 5. In the first step, the system generates what the patent calls a "**network representation**" or a "**network inventory**" that holds information about objects on the network. ('300, Fig. 5 (step 200), 6:34-35, 3:63-4:2.) These network objects may include, for example, "computers, terminals, servers, storage devices, database, routers, switches, printers, communications lines, circuits, and other telecommunications media and devices." ('300, 4:2-6.) The specification explains that "[t]he network representation may be generated automatically by a computer application or manually by a user of the system. The network

representation may include a representation of objects in the network and their relationships." ('300, 6:40-44.)

41. The "**network representation**" may be created using Extensible Markup Language (XML) or some other form of computer-readable "structured information." ('300, Abstract, 3:34-37, 4:8-10, 9:18-25.) According to the specification, using XML to encode the network representation provides a "standards compliant representation" that enables the customer to define and customize its network model. ('300, 3:34-41.) As I explained in **Part IV** above, XML was well-known and widely deployed prior to the filing of the '300 patent.

42. The second step of the process (as shown Figure 5) is to **parse** the generated network representation. ('300, Fig. 5 (step 202).) The specification does not provide any specific technical details about how the parsing step is carried out; it merely states that it occurs. ('300, *e.g.*, 6:44-45 ("In step **202**, the network representation is parsed. The parsed network representation is used to generated [sic] a network model, step **204**."), 1:44, 3:42-45.)

43. Next, the parsed network representation is used to generate a "**network model.**" ('300, Fig. 5 (step 204), 6:44-46.) The network model includes "a plurality of network objects and relationships between the plurality of network

objects." ('300, 1:44-46.) "The network model may be stored in memory for further use." ('300, 1:46-47.)

44. The specification identifies at least two ways in which the system can use the generated "**network model**." First, the system may use the network model to generate a graphical representation of the network. ('300, Fig. 5 (step 206), 6:46-49.) Second, the network model may also be used to process a "network event." ('300, Fig. 5 (steps 210, 212, 214, 216), 6:49-56.) Processing the network event may include "determin[ing] the order of operations needed to process the network event," and executing the event itself. ('300, 6:53-56, Fig. 5 (steps 216, 218).) A "**network event**," according to the specification, may include "provisioning, circuit provisioning, service provisioning, switch provisioning, rollback, and delete." ('300, 2:52-55.)

45. The term "provisioning" to persons of ordinary skill in the art generally refers to a process of making a network, computing device or other resource available to users. Provisioning is a broad concept that can involve, among other things, the physical and software configuration of the devices and assigning appropriate permission settings to users who will access them.

B. The Claims of the ’300 Patent

46. This Declaration will address claims 1, 7, 8, 10, 21 and 22. Of these six claims, only three of them—claims 1, 10 and 21—are independent, and all three recite similar subject matter. Representative claims 1, 7 and 8 recite:

> 1. A method of modelling a communications network using a computer system, the method including:
>
> [a] generating a network representation using computer-readable code, the computer-readable code representing structured information;
>
> [b] parsing the network representation;
>
> [c] generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects;
>
> [d] storing the network model in memory; and
>
> [e] processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects, and the processing further includes determining an order of operation on the one or more network objects.
>
> 7. A method according to claim **1** wherein the computer-readable code is in extensible mark-up language (XML).
>
> 8. A method according to claim **1**, further including:
>
> generating a graphical representation of the network model; and
>
> displaying the graphical representation on a display monitor.

('300, 9:41-56 (claim 1), 10:11-17 (claims 7 and 8).) The bracketed notations shown in claim 1 (e.g. "[a]," "[b]," etc.) were added and will be used below to more easily identify these claim limitations in my analysis below.

47. The other three claims addressed in this Declaration are substantially similar to claims 1, 7 and 8 listed above. Independent claim 10 is the same as claim 1 except that the first step recites "generating a network representation in extensible mark-up language (XML)," incorporating the XML requirement recited in dependent claim 7. Claims 21 and 22 are simply the "computer program product" claims corresponding to claims 1 and 8, respectively.

C. Claim Construction

48. I have been informed by counsel that invalidity involves a two-step analysis. In the first step, the scope and meaning of a claim is determined by construing the terms of that claim. In the second step, the claim so interpreted is compared against the disclosures of the prior art. Accordingly, before I address how the prior art compares to the claims of the '300 patent in **Part VI** below, I will provide my interpretation for certain terms in the claims addressed in this Declaration, from the perspective of a person of ordinary skill in the art.

49. I have been informed by counsel that a claim in an unexpired patent subject to *inter partes* review must be given its broadest reasonable construction that is consistent with the specification of the patent in which it appears, which is different from the manner in which the scope of a claim is determined in litigation. I have applied this standard in my analysis below.

1. "Network Representation"

50. In my opinion, the broadest reasonable interpretation of "**network representation**" is "**information about at least one object in the network or its relationship to the network**." I base this opinion primarily on the specification of the '300 patent, which explains:

> In one embodiment, the network representation is an XML document but may be created and/or stored in memory in any suitable format including, but not limited to, text, markup languages, programming languages, database languages, graphical representation, and any other computer-readable format. The network representation may be generated automatically by a computer application or manually by a user of the system. <u>The network representation may include a representation of objects in the network and their relationships</u>.

('300, 6:35-44 (underlining added).) The specification also provides the following list of examples of "network representation" information:

> The communications network representation may include, but is not limited to, one or more of the following elements in any suitable combination: circuit level index, circuit type identification, order of operation indication, delete circuit identification, underlying circuit index, underlying link index, delete object identification, parent circuit identification, and child circuit identification.

('300, 2:42-48 (underlining added).)

51. The categories of information identified in this list confirm that "network representation" information, under its broadest reasonable construction, is satisfied by even the most basic information about a device. For example, the passage states that a "network representation" is satisfied by simply "circuit type identification," which simply identifies the type of connection and does not necessarily indicate the other nodes to which the device is connected. In light of this broad list of exemplary "network representation" information, I have interpreted "network representation" as "information about at least one

object in the network or its relationship to the network" under its broadest reasonable construction.

2. "Network Model"

52. As explained previously, the specification and claims of the '300 patent describe a process in which a "network representation" is generated, parsed, and then used to generate a "**network model**." In my opinion, the term "network model" under its broadest reasonable construction is "**information about objects in the network and the relationships between them**."

53. This construction is supported by both the specification and claim language. The specification explains that "[n]etwork models provide a representation of the various objects that are modeled in the network inventory and also the relationships that exist between these objects." ('300, 3:20-23.) This is consistent with claim 1, which recites the step of "generating a network model using the parsed network representation, <u>the network model including a plurality of network objects and relationships between the plurality of network objects</u>."

3. "Network Event"

54. The specification and claims of the '300 patent state that once a "network model" is generated, that model can be used to process a "network

event." In my opinion, "**network event**" under its broadest reasonable construction is "**one or more operations that can be performed on or by a network or network object**." This interpretation finds support in the specification, which lists provisioning a network object or deleting it from the network as examples of network events:

> Network events may be executed using the communications network representation. <u>The network event may be selected from the group consisting of provisioning, circuit provisioning, service provisioning, switch provisioning, rollback, and delete. However, any other desired network events or received data may be processed using the present invention</u>. The network event may be received from the middleware bus. Processing of the network event may further include identifying one or more of the network objects using the network representation, and determining an order of operation on the one or more network objects using the network representation.

('300, 2:51-62 (underlining added).) As I explained previously, the term "provisioning" generally refers to the process of making a network, computing

device or other resource available to users of the network. The specification further describes network events as follows:

> In step **206**, the system determines whether a graphical representation of the network is to be generated. . . If no, then the system determines whether an event is to be processed, step **210**. . . If yes, then the system identifies the needed objects in the network model, step **214**. In step **216**, the system determines the order of operations needed to process the network event. In step **218**, the system then executes the event as required.

('300, 6:46-56.) Based on this description, I have interpreted "network event" under its broadest reasonable construction as "one or more operations that can be performed on or by a network or network object."

VI. APPLICATION OF THE PRIOR ART TO THE CLAIMS OF THE '300 PATENT

55. I have reviewed and analyzed the prior art references and materials listed in **Part II.B** above. In my opinion, each limitation of claims 1, 7, 8, 10, 21 and 22 is disclosed or suggested by the following prior art references:

Exhibit No.	Prior Art Reference
1003	U.S. Patent App. Pub. No. 2002/0161883 to David Matheny et al.
1004	Elliotte Rusty Harold et al., *XML in a Nutshell* (2001)

Exhibit No.	Prior Art Reference
1005	U.S. Patent No. 5,796,951 to John F. Hamner et al.
1006	U.S. Patent No. 6,256,635 to Yan Arrouye et al.
1007	U.S. Patent No. 5,717,934 to Ronald L. Pitt et al.

56. Counsel has informed me that these references qualify as prior art to claims 1, 7, 8, 10, 21 and 22 because they were all published more than one year before January 8, 2004, the filing date of the application for the ’300 patent. Before applying the art to the claims of the ’300 patent, I will provide a brief summary of these prior art references.

Matheny [Ex. 1003]

57. Matheny, entitled “System and Method for Collecting, Aggregating, and Coalescing Network Discovery Data,” discloses a technique for identifying devices on a computer network and collecting information about them. (Matheny, Abstract, ¶ 0011.) Matheny discloses a system that includes “discovery agents” (**106**) for gathering and collecting information about the devices on a network, using standard network management protocols. (Matheny, ¶¶ 0011-0014, 0021.) These discovery agents (**106**) generate and store a series of “discovery” files that contain information that was collected from each node

discovered on the network, and describe how the discovered nodes relate to each other. (Matheny, ¶¶ 0021, 0022.) These "discovery" files may be stored in XML format. (Matheny, ¶¶ 0021, 0022, 0016.) As shown below, these XML files disclose the "network representation" claimed in the '300 patent.

58. Once these XML discovery files have been generated, a network manager (**104**) in a network management (**102**) performs a process in which the files are parsed in order to "coalesce" their information into an aggregated file, known as a "discovery document," which can be incorporated into a database. (Matheny, ¶¶ 0024, 0025, 0028.) This "discovery document" corresponds to the "network model" claimed in the '300 patent, as shown below.

<u>XML in a Nutshell</u> [Ex. 1004]

59. XML in a Nutshell is a textbook that describes aspects of the Extensible Markup Language (XML). This reference explains that, in order for an application to use the information in an XML document or store it in a database, the system must parse the XML to extract the data. (XML in a Nutshell, at 7; *id.* at 5 ("Software on the server side receives the XML data, parses it, and stores it in the database.").) I have relied on this reference for a narrow purpose in combination with Matheny of showing disclosure of parsing XML files.

Hamner [Ex. 1005]

60. Hamner is a patent that describes a network management system similar in many respects to Matheny. Hamner differs from Matheny in that it provides a more detailed description of the "nuts and bolts" of a prior art network management system, including the user interface that one would employ to manage and configure a network. (Hamner, Fig. 2A.) I have relied on Hamner for claim limitations relating to "processing a network event," as Hamner discloses this aspect of the claims in more detail than Matheny alone. For example, Hamner discloses a system that identifies particular "tasks" that can be performed on a network device. (Hamner, 4:18-34, Fig. 2A (task window **202**).) The tasks are carried out by "script" programs that can be executed on the affected devices. (Hamner, 10:66-11:7.) The tasks that may be performed include running reports, executing a virus scan, rebooting selected computers on the network, and other troubleshooting and monitoring functions. (Hamner, 3:51-56, 11:6-7.)

Arrouye [Ex. 1006]

61. Arrouye discloses a network management system similar in many respects to the one in Hamner. I have relied on Arrouye for the final limitation of

each independent claim, which requires that the processing step also determine "an order of operation on the one or more network objects." Arrouye discloses a network management system in which, like Hamner, a network administrator can execute a task on a particular network object (e.g. computer on the network) using a script program. (Arrouye, Abstract, 9:54-59, 16:57-63.) Arrouye specifically discloses that a script program enables "a computer to execute a sequence of instructions or commands," thus disclosing the claimed order of operation feature in combination with Hamner. (Arrouye, 2:51-58.)

Pitt [Ex. 1007]

62. Finally, Pitt is a patent that describes a technique for establishing a series of rules for shutting down a computer network or an individual device on a network. (Pitt, Abstract, 1:21-26.) I have relied on Pitt as an alternative disclosure for the final limitation of relating to the determination of "an order of operation on the one or more network objects." Pitt discloses a technique for established a preprogrammed sequence of operations for shut-down operations that avoids data loss and other problems. (Pitt, 5:59-63.)

63. My declaration has applied the teachings of Matheny, *XML in a Nutshell*, Hamner, Arrouye and Pitt to claims 1, 7, 8, 10, 21 and 22 of the '300

patent. In my opinion, these references provide sufficient detail to enable a person of ordinary skill in the art to practice the limitations of the claims to which they apply without undue experimentation. As explained in **Part III** above, the technological underpinnings of the challenged claims were firmly in place well before January 2004. Network management tools that would create network models, and process network events, were-known. The Background section of Hamner, for example, acknowledged that various software programs for managing networks were on the market as of December 1995. (Hamner, Ex. 1005, 1:20-24.) XML and XML parsers were already well-known by no later than the 2001 publication of *XML in a Nutshell*. (XML in a Nutshell, Ex. 1004, at 5-6.) Accordingly, by January 2004, one of ordinary skill in the art could have made what is described in claims 1, 7, 8, 10, 21 and 22 based on the disclosures in Matheny, *XML in a Nutshell*, Hamner, Arrouye and Pitt, without undue experimentation.

A. Claims 1, 7, 8, 10, 21 and 22 Are Obvious Over Matheny in view of *XML in a Nutshell*, Hamner and Arrouye

1. Each Limitation of Claim 1 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

64. The preamble of claim 1 recites a "[a] method of modelling a communications network using a computer system," comprising a series of steps. Matheny discloses such a method. As explained in more detail in the claim limitations that follow, Matheny further discloses generating a model of a communications network. Matheny discloses a technique for collecting information about each node on the network, and how those nodes are related to each other. (Matheny, ¶¶ 0021, 0022, 0024, 0028.)

(a) "generating a network representation using computer-readable code, the computer-readable code representing structured information"

65. The first limitation of claim 1 recites the step of "<u>generating a network representation using computer-readable code, the computer-readable code representing structured information</u>." As I explained in **Part V** above, the term "network representation" should be understood as "information about at least one object in the network or its relationship to the network." The specification does not impose any specific requirements on how the "generating"

step is carried out. In fact, it explains that "[t]he network representation may be generated automatically by a computer application or manually by a user of the system." ('300, 6:40-42.)

66. Matheny discloses generation of the claimed "network representation" using structured information, in particular, Extensible Markup Language (XML) files. The "network representation" in Matheny takes the form of XML files generated by the system that describe the devices on the network and their relationship to other devices. Matheny refers to these XML files as "**discovery files**," which include (1) "**collected data**" files that contain the information that was collected from each node discovered on the network; and (2) a "**relationship**" file that describes how the discovered nodes relate to each other. (Matheny, ¶ 0021, 0022.) These discovery files fully disclose the "**network representation**" recited in claim 1 of the '300 patent.

67. In particular, Matheny describes a process known as "discovery" in which the system identifies or gathers information about devices on the network. (Matheny, ¶ 0002, 0019, 0020.) A "discovery agent" (**106**) in Matheny locates a command file that instructs the agent how the discovery process should be

carried out. (Matheny, ¶¶ 0019, 0020, Figure 3 (steps 302, 304, 306).) The discovery agent then collects information from each device:

> The discovery agent **106** collects data from each device that it discovers (block **310**), and places the collected data in a file created for that device in the discovery directory (block **312**). The discovery agent may name the files in such a way as to avoid duplicates, for example, by using the device's IP address as a filename. Each file may be generated as a valid XML file with a tag line followed by the tag <node>. The terminating </node> may be omitted in order to enable aggregator agents, called later in the operation 300, to append to the file.

(Matheny, ¶ 0021 (underlining added).) In addition to the collected data files, the discovery agent (**106**) also generates an XML "**relationship"** file:

> The discovery agent may also create a relationship file, which indicates how the discovered nodes relate to each other. This file may be named using the request identifier from the original command file. In the example above, a relationship file may have the name "/somepath/command/1234567/1234567.xml".

(Matheny, ¶ 0022.)

68. Both the "collected data" and "relationship" files in Matheny, as shown above, are XML files. Matheny explains that XML may be used "as the communication language between agents and the discovery manager during the discovery operation." (Matheny, ¶ 0016.) "When the different components communicate, they do so by passing XML files to each other." (Matheny, ¶ 0016.)

69. The generation of the discovery files in Matheny, *e.g.* the XML files containing collected data and relationship information, clearly satisfies the step of "generating a network representation using computer-readable code, the computer-readable code representing structured information." The collected data and relationship files contain information about objects in the network, as well as how those objects relate to each other.

70. These XML files also qualify as "computer-readable code representing structured information" as recited in claim 1. The specification of the '300 patent, in fact, expressly identifies XML as an example of "computer-readable code" and "structured information." (*See, e.g.*, '300, *e.g.* Abstract ("The computer-readable code may be any suitable language or instructions for representing structured information such as, for example, extensible mark-up language (XML)."), 10:11-12 (claim 7: "A method according to claim 1 wherein the

computer readable code is in extensible mark-up language (XML).") (underlining added to both).) Matheny therefore discloses this limitation.

(b) "parsing the network representation"

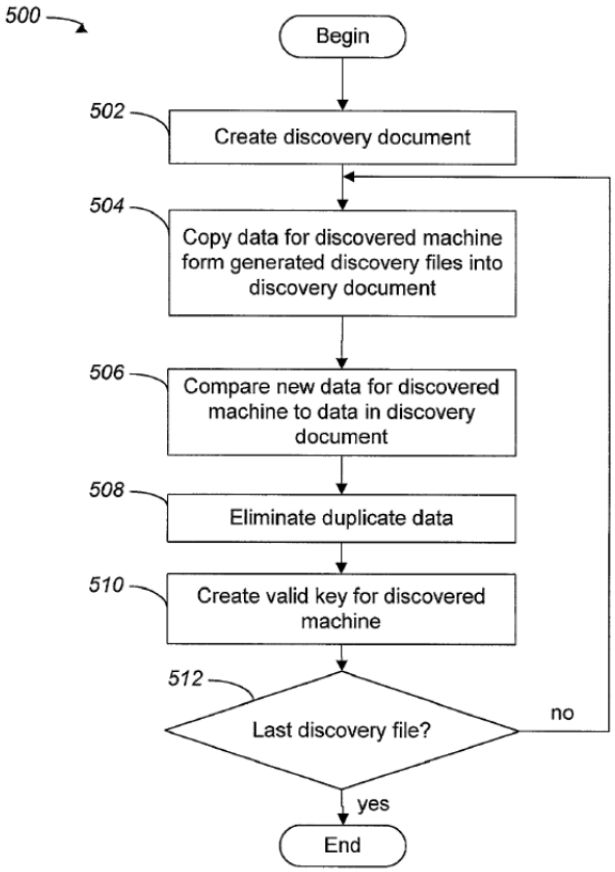
71. The next limitation of claim 1 requires the step of "parsing the network representation." Matheny discloses that the network representation – the XML discovery files discussed above – is parsed.

72. In particular, after the discovery agent (**106**) in Matheny finishes collecting data from the network devices, the network manager (**104**) performs an aggregation process in which the XML "discovery files" are parsed, resulting in "coalescing" the information into a discovery document that may be incorporated into the database. (Matheny, ¶¶ 0023, 0024, 0028.) As explained in Matheny:

> [0023] When the discovery agent completes collecting data from all available devices in the designated address range or subnet (block **314**), an aggregator operation **400** according to an embodiment is performed.
>
> * * *
>
> [0024] FIG. 5 is a flowchart describing a coalescing operation **500** according to an embodiment. After the first aggregator agent completes, the network manager **104** may create a discovery document and begin to

> coalesce the discovered and aggregated information (block **502**). The discovery document may be created with a top-level node named, for example, <i-discover>. The network manager **104** loops through the information in all of the discovery files for the discovered devices generated and appended by the various discovery and aggregator agents and copies the data into the discovery document (block **504**).
>
> [0025] In the process of copying the data, new data may be compared to previously copied data (block **506**) and any duplicate data may be eliminated (**408**).

(Matheny, ¶ 0023-0025 (underlining added).) Figure 5, described in the excerpt above, is shown below:



(Matheny, Fig. 5.)

73. Matheny does not expressly state that the network manager (**104**) "parses" the XML in the discovery files, but one of ordinary skill in the art would have understood that the process outlined above involves parsing. As I explained in **Part IV** above, XML encodes data elements and contents within human-readable tags. In order to locate the information within the XML files and copy it into the discovery document, the network manager (**104**) in Matheny must parse the XML in the discovery files to extract the data from the tagged XML data.

74. Without parsing the XML data, the network manager (**104**) in Matheny could not "loop[] through the information in all of the discovery files," allow "new data [to] be compared to previously copied data," or "cop[y] the data into the discovery document." (Matheny, ¶¶ 0024, 0025.) One of ordinary skill in the art, therefore, would have understood these processes in Matheny as encompassing parsing of the XML discovery files, and therefore, disclosing the step of "parsing the network representation" in claim 1.

75. To the extent there is any question as to whether Matheny necessarily discloses "parsing the network representation," this step is obvious in view of **XML in a Nutshell** **[Ex. 1004]**. Chapter 1 of *XML in a Nutshell*, entitled "Introducing XML," confirms that parsing is fundamental to the use of an XML file by an application program:

> Programs that actually try to understand the contents of the XML document, that is, do not merely treat it as any other text file, use an *XML parser* to read the document. The parser is responsible for dividing the document into individual elements, attributes, and other pieces. It passes the contents of the XML document to an application piece by piece.

(XML in a Nutshell, at 7.)

76. It would have been obvious to one of ordinary skill in the art to combine Matheny with *XML in a Nutshell*, predictably resulting in the network modeling system of Matheny in which the network manager (**104**) includes an XML parser that parses the XML discovery files (the "network representation"). Under this combination, an XML parser would parse the discovery files in Matheny and "pass[] the contents of the XML document to [network manager (**104**)], piece by piece." (XML in a Nutshell, at 7.) One of ordinary skill in the art would have regarded this as a straightforward and trivial combination. As I mentioned previously, and *XML in a Nutshell* confirms, parsing an XML file is a prerequisite to the use of the data by an application program. (*Id.*)

77. *XML in a Nutshell* also provides several motivations to combine. It explains that the application that receives data from the XML parser may be any type of application, including a database that stores the XML data into a new record. (XML in a Nutshell, at 7-8.) As explained in that reference: "Software on the server side <u>receives the XML data, parses it, and stores it in the database</u>." (*Id.* at 5 bottom paragraph, underlining added).) This is analogous to the process in Matheny in which the network manager (**104**) extracts data from the XML

discovery files and stores it into a "discovery file," which is later copied into a "discovery database." (Matheny, ¶ 0024, 0025, 0028, Fig. 3 (step 316).)

78. Finally, *XML in a Nutshell* explains that XML and XML parsers present benefits over other types of data formats (such as tab-delimited text files). "The difference is that XML parsers present you with the data in a much more convenient format to work with and do more of the work for you before you have to resort to your own custom code." (XML in a Nutshell, at 5 (top part of page).) For all of these reasons, one of ordinary skill in the art would have had ample reason to combine Matheny with the parsing disclosures in *XML in a Nutshell*.

(c) "generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects"

79. Matheny also discloses the step of generating a network model using the parsed network representation. The "**network model**" in Matheny takes the form of a "**discovery document**" that identifies the network objects that were discovered, and the relationships between them.

80. In particular, the network manager (**104**) performs a process of "coalescing" the data from the XML discovery files (the "network representation")

to generate a "discovery document," which corresponds to the "network model" recited in the '300 patent. As explained in Matheny:

> [0024] FIG. 5 is a flowchart describing a coalescing operation **500** according to an embodiment. After the first aggregator agent completes, the network manager **104** may create a discovery document and begin to coalesce the discovered and aggregated information (block **502**). The discovery document may be created with a top-level node named, for example, <i-discover>. The network manager **104** loops through the information in all of the discovery files for the discovered devices generated and appended by the various discovery and aggregator agents and copies the data into the discovery document (block **504**).

(Matheny, ¶ 0024.)

81. As explained previously, these teachings from Matheny, alone or in combination with *XML in a Nutshell*, disclose parsing the XML discovery files. *XML in a Nutshell* confirms that in order to store data from an XML file into a database, the XML must be parsed. (XML in a Nutshell, at 5 ("Software on the server side receives the XML data, parses it, and stores it in the database.") (bottom paragraph, underlining added).) Because the "discovery document" was

generated by parsing and then copying data from the XML discovery files, Matheny and *XML in a Nutshell* disclose "generating a network model using the parsed network representation" as claimed.

82. The discovery document in Matheny also "<u>includ[es] a plurality of network objects and relationships between the plurality of network objects</u>," as recited in the latter portion of claim 1**[c]**. As explained above, the XML discovery files generated by the discovery agents (**106**) include this information:

- "The discovery agent **106** collects <u>data from each device that it discovers</u> (block **310**), and places the collected data in a file created for that device in the discovery directory (block **312**)." (Matheny, ¶ 0021 (underlining added).)

- "The discovery agent may also create <u>a relationship file</u>, which <u>indicates how the discovered nodes relate to each other</u>." (Matheny, ¶ 0022 (underlining added).)

83. Matheny explains that the network manager (**104**) copies the information from these discovery files into the "discovery document," *i.e.* the claimed "network model." (Matheny, ¶ 0024 ("The network manager **104** <u>loops through the information in all of the discovery files for the discovered devices</u> generated and appended by the various discovery and aggregator agents <u>and copies the data into the discovery document</u> (block **504**).") (underlining added).) The "discovery document" in Matheny therefore includes the information from

the discovery files, which includes the "data from each device" and "how the discovered nodes relate to each other." (Matheny, ¶¶ 0021, 0022.) The network model therefore includes "a plurality of network objects and relationships between the plurality of network objects," as recited in claim 1**[c]**.

(d) "storing the network model in memory"

84. Matheny also discloses the step of "storing the network model in memory." As explained previously, Matheny discloses the creation of a "discovery document" generated by the network manager (**104**), into which information from the discovery files is copied. Matheny further discloses that this "discovery document" (the claimed "network model") may be stored in a discovery database:

> [0028] After the aggregator operation **400** has been performed on all files in the discovery directory created by the last-called discovery agent **106**, the discovery document may be copied to the discovery database (block **316**), and the discovery directory and command files generated for the discovery operation **300** removed (block **318**).

(Matheny, ¶ 0028 (underlining added); *see also* Matheny, claim 26 ("…further comprising instructions that cause the machine to: store the discovery document

in a discovery database…") (underlining added).) Matheny therefore discloses the step of "storing the network model in memory."

(e) "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects…"

85. Matheny and Hamner disclose the step of "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects." As I explained in **Part V** above, the term "network event" should be understood as meaning "one or more operations that can be performed on or by a network or network object."

86. As explained above, the network manager (**104**) in Matheny generates a "network model" in the form of a "discovery document" that is stored in a discovery database. (Matheny, ¶¶ 0024, 0028.) This information can be used to process a network event:

> [0002] One type of network management tool is the discovery agent. These agents gather information from devices in the network. This information may be used to evaluate network performance and possible faults, as well provide information needed to reconfigure the network.

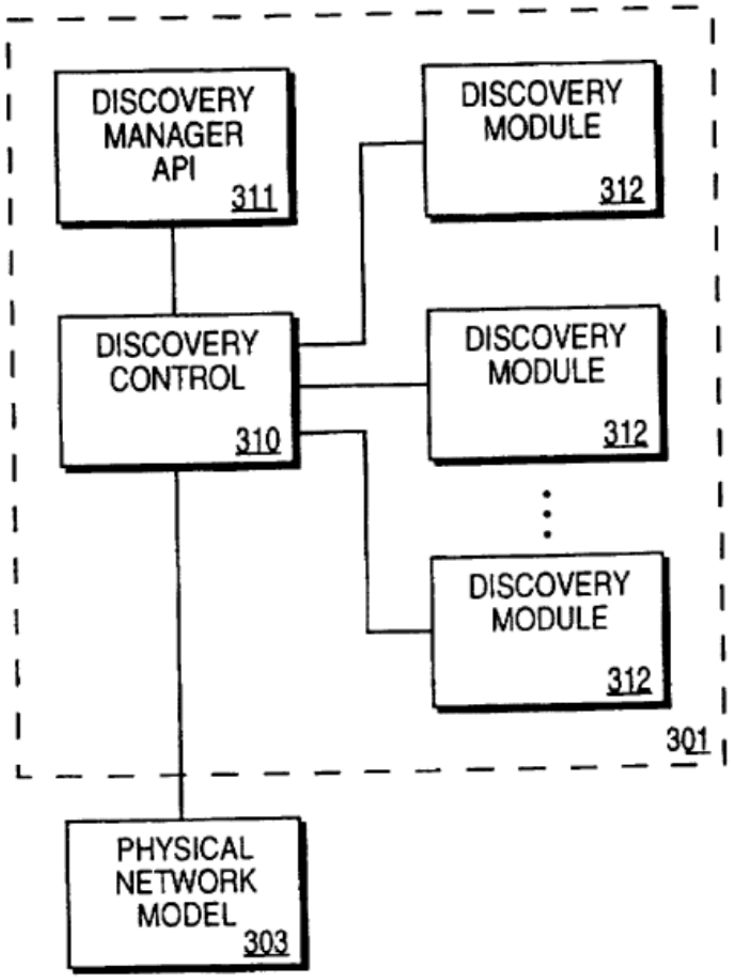
(Matheny, ¶ 0002 (underlining added).)

87. Matheny therefore discloses "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects." One of ordinary skill in the art would have understood that the acts of "evaluat[ing] network performance and possible faults," at a minimum, included identifying one of more devices in the network. In my opinion, the statements from the Background of Matheny quoted above disclose the claimed processing step, but Matheny does not provide further details. (Matheny, ¶ 0002.) This is because Matheny focuses on *generation* of a network model and not how the model is thereafter put to use.

88. To the extent there is any doubt as to whether Matheny alone discloses the step of "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects," this step is disclosed by Matheny in view of Hamner.

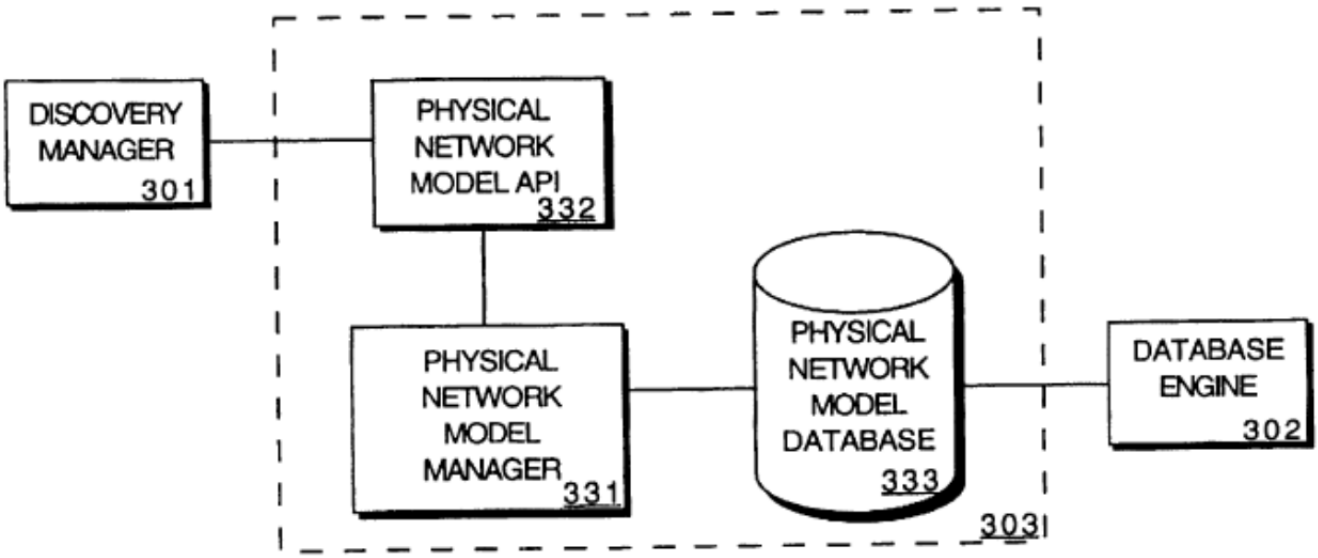
Overview of Hamner

89. **Hamner** discloses a computer-implemented network management system similar in many respects to the one described in Matheny. (Hamner, Abstract.) Hamner discloses a "discovery manager" (**301**) that includes a number of "discovery modules" (**312**) as shown in Figure 4 below. (Hamner, Fig. 4, 5:44-55.)



(Hamner, Fig. 4.)

90. These "discovery modules" (**312**) perform the exact same function as the "discovery agents" (**106**) of Matheny – to "poll" network devices on the network to gather information about them. (Hamner, 5:51-54; *compare with* Matheny, ¶ 0011.) The system in Hamner then uses the information collected from the discovery modules (**312**) to construct a physical network model (**303**). (Hamner, 5:55-57 ("When a device is found by a discovery module **312**, that module then adds information about the device to the physical network model **303**.").) "<u>The physical network model (PNM) **303** represents all known pieces of the managed network **10** and how those pieces interrelate.</u>" (Hamner, 7:43-45 (underlining added).) The network model is stored in a "physical model database" (**333**), shown in Figure 6 below. (Hamner, 7:45-51.)



(Hamner, Fig. 6.)

91. Hamner provides network management functions through a "Management Console" shown in Figure 2A below, which lists the devices on the network and the tasks that can be performed on the devices:

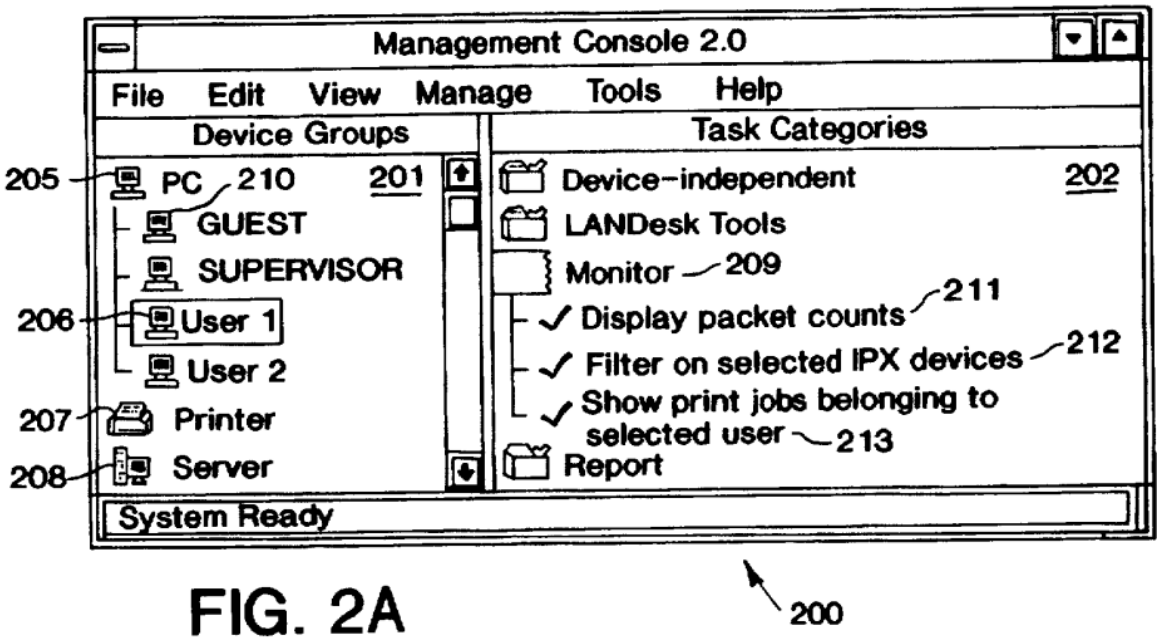


FIG. 2A

(Hamner, Fig. 2A.)

92. Figure 2A above includes a device window (**201**) on the left that shows the devices on the network organized based on the group in which they belong (*e.g.* the "PC" device group (**205**), the "Printer" device group (**207**), or "Server" device group (**208**)). (Hamner, 4:3-16.) The right side of Figure 2A includes a task window (**202**) that displays tasks and categories of network

management tasks that can be performed on the currently-selected network device. (Hamner, 4:18-32.) "The user can cause any displayed task to be performed upon a device." (Hamner, 4:33-34.)

Matheny and Hamner Disclose the "Processing" Step

93. The combination of Matheny and Hamner discloses the step of "processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects," as recited in claim 1**[e]**. The "**network events**" correspond to the "tasks" in Hamner, including the ones shown in the task window (**202**) of Figure 2A above. As explained above, the term "network event" should be understood as "one or more operations that can be performed on or by a network or network object." The "tasks" in Hamner clearly satisfy this definition.

94. "Tasks include functions for troubleshooting, monitoring, and reporting on devices." (Hamner, 11:6-7, 3:49-51.) "Some examples of tasks which might be performed on various devices in a network are: viewing the screen of a particular PC; displaying packet counts; running a report; executing a remote virus scan; rebooting selected workstations; displaying print jobs; or, displaying non-functioning printers." (Hamner, 3:51-56.) "The user can cause any

displayed task to be performed upon a device." (Hamner, 4:33-34.) Each one of these tasks qualifies as a "network event."

95. Hamner also discloses "processing a network event using the network model," as recited in the claim. The "network model" corresponds to the "physical network model" (PNM) in Hamner, which is stored in the "physical model database" (**333**). (Hamner, 7:43-53.) The tasks that can be performed (the "network events") are obtained from the network model in Hamner. For example, "[i]n determining the executable tasks in the network and the devices on which such tasks can be performed, the task manager **305** makes use of the PNM [physical network model] schema described with reference to FIG. 7." (Hamner, 11:8-11 (underlining added), 11:18-21.) "In the preferred embodiment, available tasks are stored in the physical model database **333**." (Hamner, 11:20-21 (underlining added), 8:47-61 (explaining how task information for network devices is stored in the physical network model database (**333**).) Because the "network model" is used to determine and store the available tasks, Hamner discloses using the "network model" to process network events.

96. Finally, Hamner discloses that the processing of the network events "includes identifying one or more network objects of the plurality of network

objects." This is because Hamner discloses that the "tasks" could include network events that would identify one or more network objects. For example, the tasks include "displaying non-functioning printers," "viewing the screen of a particular PC," "running a report" or "rebooting selected workstations." (Hamner, 3:51-56.) The tasks in Hamner can also include "troubleshooting, monitoring, and reporting on devices." (Hamner, 11:6-7.) Any one of these tasks would result in "identifying one or more network objects," as recited in the claim.

97. Additionally, the identification of network objects is shown in Figure 2A reproduced above, which displays (in graphical form) network devices such as workstations, servers and printers, organized by device group. (Hamner, 4:5-7 ("Devices and device groups in the network are represented in the device window **201** [of Figure 2A]").) The system of Hamner uses the physical model database (**333**) to identify the device groups and the devices in them. (Hamner, 9:61-67, 9:64-66 ("The group view generator **341** obtains group definitions from the physical model database **333**…").) Hamner therefore discloses identifying one or more network objects as recited in the claim.

Rationale for Combining Matheny and Hamner

98. It would have been obvious to one of ordinary skill in the art to combine the teachings of Matheny and Hamner. This would have predictably resulted in a system in which the network management system of Hamner used the "network model" in Matheny (the "discovery document" was generated by aggregating and coalescing the XML discovery files) to process "network events" and identify objects on the network.

99. Matheny and Hamner are analogous references in the same field of network management and device discovery, and the references share numerous similarities and parallels. For example, the "discovery modules" (**312**) in Hamner perform the exact same function as the "discovery agents" (**106**) in Matheny:

- **Hamner**: "The various discovery modules **312** poll the network **10** to find specific types of devices. Each discovery module is a specialist in finding a certain type of device, although there may be overlap in the particular devices found by different discovery modules." (Hamner, 5:51-54.)

- **Matheny**: "The discovery agents **106** collect information from targeted network devices during a discovery operation (described below), for example, by polling devices in a certain range of addresses or in a particular subnet." (Matheny, ¶ 0011.)

These and other similarities reflect the fact that Matheny and Hamner are directed to solving the same network discovery and management problems using

similar technological solutions.

100. As noted previously, Matheny discloses the idea of processing network events using a network model, but does not provide details about how this is carried out. (Matheny, ¶ 0002 ("This information may be used to evaluate network performance and possible faults, as well as provide information needed to reconfigure the network.").) This omission is simply a consequence of the fact that Matheny chose to focus on the *creation* of the network model itself. One of ordinary skill in the art reading Matheny would have understood that Matheny fully contemplated subsequent use of the network model to process a network event. (Matheny, ¶ 0002.) This is such a basic function of network management systems by January 2004 that one of ordinary skill in the art would not have been surprised in any way by its omission from Matheny. One of ordinary skill in the art would understand that a network management system like Hamner could take over where Matheny leaves off by using the "network model" to process events.

101. Hamner also provides express motivations to combine its network management services with a system like Matheny. Hamner explains that "[t]he management services are designed from a flexible, device-oriented perspective in order to provide the user with information on the layout of the network, tasks

that can be performed, and the devices on which each particular task can be performed." (Hamner, 12:22-27.) One of ordinary skill in the art would have understood that the system in Hamner could be used to provide a user interface that makes use of the network model generated by the system of Matheny.

(f) "…and the processing further includes determining an order of operation on the one or more network objects"

102. The final limitation of claim 1 requires that the processing of the network event "further includes determining an order of operation on the one or more network objects." As explained below, this step corresponds in Hamner to identifying a "script," which defines a sequence of operations to be performed in a network device. In particular, I explained in the preceding claim limitation that Hamner provides a database containing one or more "tasks" that can be performed on devices in the network. (Hamner, 11:8-11, 11:20-21, 8:47-61.) Hamner further describes a "task" is a script that can be executed on a device:

> A task consists essentially of an atomic script and any associated parameters. A parameter associated with a task may be a device upon which a task will be performed. Tasks include functions for troubleshooting, monitoring, and reporting on devices

(Hamner, 11:3-7 (underlining added).)

103. The term "**script**" in computer science refers to a sequence of instructions to perform a specific function. (See *Microsoft Computer Dictionary* pp.422-423 (3rd ed. 1997) **[Ex. 1008]** (defining "script" as: "A program consisting of a set of instructions to an application or utility program.").) Most "scripts" are written using a script programming language (such as shell scripting languages (e.g., ksh or csh)) that is simpler and easier to understand than conventional programming languages such as C or C++.

104. Scripts are used to perform a wide variety of tasks across almost all aspects of computing. For example, web browsers, productivity applications (such as the Microsoft Office suite), network management systems and numerous other types of software programs allow developers and end-users to write "scripts" to provide additional functionality. In fact, scripting languages are often designed to be simple enough so end-users with no technical education and little or no prior programming experience can write useful script programs.

105. One of ordinary skill in the art would have understood that identifying or executing a "**script**" on a network device, as disclosed in Hamner, discloses the step of "<u>determining an order of operation on the one or more</u>

network objects," as recited in claim 1**[f]**. This is because a script (like any computer program) includes instructions that must be executed in the particular order specified in the script. Identifying a script to be executed on a device therefore satisfies the step of "determining an order of operation" for that device.

106. It is elementary computer science that computer programs such as scripts consist of a sequence of instructions that must be executed or carried out in a predetermined order, or the computer program will not function correctly. The precise order in which the instructions are carried out depends on the order in which these instructions appear in the script file, as well as the instructions themselves (which can include branching instructions transferring control to other instructions). In any event, the script will dictate the precise order in which the operations must occur. Hamner's disclosure of using a "script" to process a network event for a device, therefore, discloses determining an order of operation on the device.

107. Although I believe that one of ordinary skill in the art would find the "order of operation" limitation to be satisfied by the disclosure of "scripts" in Hamner, the "order of operation" is further disclosed by **Arrouye** **[Ex. 1006]**.

Arrouye Further Discloses the "Order of Operation" Feature

108. Arrouye, entitled "Method and Apparatus for Configuring a Computer Using Scripting," discloses a network and computer management system in which a script program can be used to configure a remote computer or other device. (Arrouye, Abstract, 9:54-59, 16:57-63.) Arrouye confirms the point made above – that a "script" specifies a sequenced or ordered series of operations on a device:

> To enable a computer to execute a sequence of instructions or commands, a computer program is utilized. One type of computer program that provides for the automation of a task or a series of tasks is called a script. A script can be used to perform simple tasks such as a file back-up, or more complex tasks such as adding capability to a program or presentation. A scripting language is used to create a script.

(Arrouye, 2:51-58 (underlining added).) The phrase "sequence of instructions" is commonly used in computer science to describe computer programs and scripts, and to indicate that their instructions must be executed in a particular order.

109. Arrouye further discloses that scripts can be used to "perform a plurality of events *sequentially*" on a device on a computer network:

> One embodiment of the invention provides the ability to perform remote scripting. Remote scripting allows an administrator to set up a script on the administrator's computer to perform a plurality of events sequentially on the administrator's computer or on a computer connected to the network. In addition, a script can be stored remotely and an administrator or user may command the script to run. The ability to run a script to perform a plurality of events on the network enables a network administrator to automatically custom configure each network user's computer from a remote location.

(Arrouye, 9:54-64 (underlining added).)

110. In my opinion, the disclosures in Hamner and Arrouye leave no doubt that selecting a task in Hamner satisfies the step of "determining an order of operation on the one or more network objects," as recited in the claim. As noted above, Hamner explains that "[t]he user can cause any displayed task to be performed upon a device." (Hamner, 4:33-34.) Because Hamner states that "[a] task consists essentially of an atomic script" (Hamner, 11:3-4), selecting the task for execution results in determining an order of operation on the one or more network devices. The order of operation, as explained above, is specified by the

"sequence" of instructions or commands in the script, which are performed "sequentially" on the affected device. (Arrouye, 2:51-58, 9:54-60.)

Rationale for Combining Arrouye with Matheny and Hamner

111. It would have been obvious to one of ordinary skill in the art to adapt the teachings of Arrouye to the network management systems disclosed in Hamner and Matheny. This would predictably have resulted in the network management systems of Hamner and Matheny in which the processing a network event using a "script" included determining an order of operations on the one or more network objects.

112. As explained above, Arrouye confirms that computer programs and scripts specify operations *and* the order in which they must be carried out. This detail is so elementary and trivial that one of ordinary skill in the art would not even have had to rely on the disclosures of Arrouye to understand it. In any event, Arrouye is also fully combinable with Hamner and Matheny because all three references are analogous and in the same field of managing and configuring network devices. Hamner and Arrouye, as explained above, are similar systems that both allow the use of "scripts" to process network events on devices in the

network. This reflects the fact that both references are directed to common problems that are addressed with similar solutions.

113. Arrouye also provides express motivations to apply its remote scripting teachings, by explaining that remote scripting can ease network device configuration and prevent potential provisioning errors:

> The ability to run a script to perform a plurality of events on the network enables a network administrator to automatically custom configure each network user's computer from a remote location. This capability eliminates the need to physically and manually adjust each computer that is connected to the network. If a network contains several hundred computers and each has a static IP address that needs to be changed, the network administrator saves considerable time by merely creating a script to adjust the IP address on each user's computer. This also prevents possible errors if each computer user were required to manually adjust the computer's configuration

(Arrouye, 9:60-10:5.) For all of these reasons, a person of ordinary skill in the art would have had ample motivation to apply Arrouye's remote scripting teachings to the systems of Hamner and Matheny.

2. Each Limitation of Claim 7 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

114. Claim 7 depends from claim 1 and recites: "A method according to claim **1** wherein the computer-readable code is in extensible mark-up language (XML)." As explained for claim 1**[a]** above, Matheny discloses that the discovery files generated by the discovery agents (**106**) – the claimed network "representation using computer-readable code" – are XML files. (Matheny, ¶ 0016 ("[XML] may be used as the communication language between agents and the discovery manager during the discovery operation. When the different components communicate, they do so by passing XML files to each other."), ¶¶ 0021, 0022 (collected data and relationship information are stored in XML files).) For the same reasons as claim 1, therefore, claim 7 is disclosed by the Matheny in view of *XML in a Nutshell*, Hamner and Arrouye.

3. Each Limitation of Claim 8 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

115. Claim 8 also depends from claim 1 and adds the additional steps of "<u>generating a graphical representation of the network model</u>; and <u>displaying the graphical representation on a display monitor</u>." Each of these limitations is fully disclosed by Hamner, as explained in connection with claim 1. For example, the

network "Management Console" shown in Figure 2A provides a graphical representation of the network model shown on a display monitor:

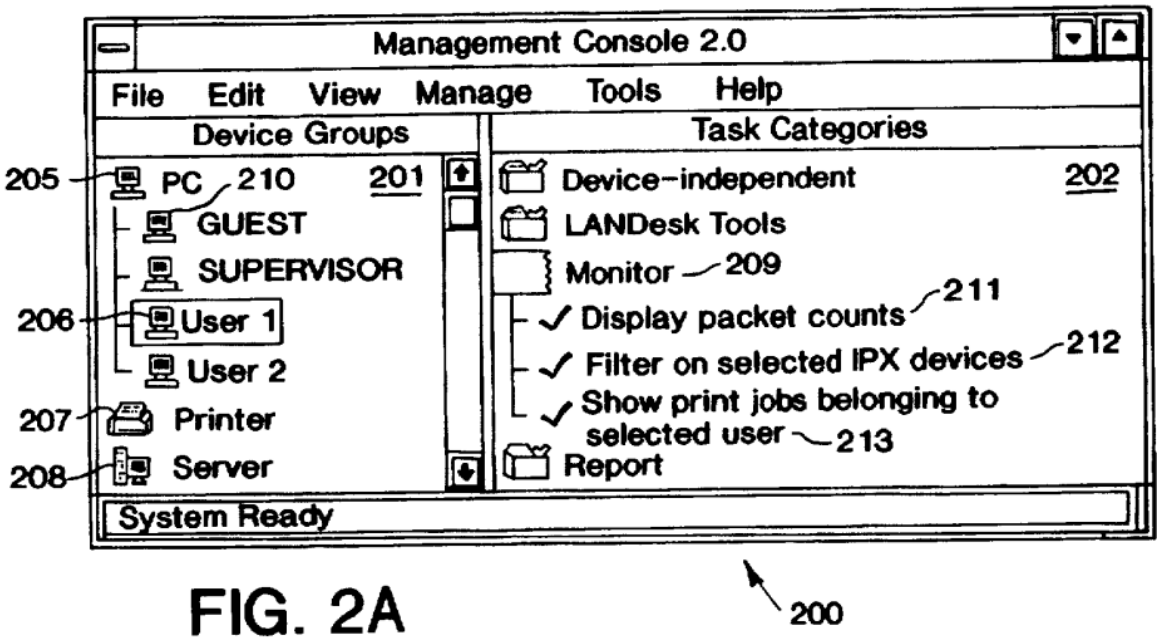


FIG. 2A

(Hamner, Fig. 2A.)

116. The exemplary system in Figure 2A of Hamner includes a device window (**201**) on the left that shows the various devices on the network organized based on the group in which they belong (*e.g.* the "PC" device group (**205**), the "Printer" device group (**207**), or "Server" device group (**208**)). (Hamner, 4:3-16.) Figure 2A shows the network modelled into a "tree" hierarchy allowing the user to click on a particular device group, such as the PC device group (**205**) in Figure 2A, to view all of the devices that fall within the selected group. (Hamner,

4:10-16.) The system in Hamner also contemplates "schematic, geographic or other views of the layout of the network, in addition to or instead of a tree hierarchy view." (Hamner, 4:53-56.) For the same reasons as claim 1, therefore, claim 8 is disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye.

4. Each Limitation of Claim 10 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

117. Independent method claim 10 is materially identical, for purposes of my analysis, to independent claim 1 and dependent claim 7. A side-by-side chart showing these claims is provided below (with common language underlined):

Claim 1	Claim 10
1. A method of modelling a communications network using a computer system, the method including:	10. A method of modelling a communications network using a computer system, the method including:
generating a network representation using computer-readable code, the computer-readable code representing structured information;	generating a network representation in extensible mark-up language (XML);
parsing the network representation;	parsing the network representation;
generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects;	generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects;
storing the network model in memory; and	storing the network model in memory; and

Claim 1	Claim 10
processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects, and the processing further includes determining an order of operation on the one or more network objects.	processing a network event using the network model, wherein processing the network event includes identifying one or more of the plurality of network objects, and determining an order of operation on the one or more of the the [sic] plurality of network objects.

118. As the table above shows, the only material difference between claim 10 and claim 1 is that the first limitation of claim 10 recites "generating a network representation in extensible mark-up language (XML)," incorporating the limitation from dependent claim 7 described above. As explained for claim 1**[a]** and claim 7 above, Matheny discloses that the discovery files generated by the discovery agents (**106**) – the claimed network "representation using computer-readable code" – are in XML. (Matheny, ¶¶ 0016, 0021, 0022.) For the same reasons as claims 1 and 7, therefore, claim 10 is disclosed by Matheny in view of *XML in a Nutshell*, Hamner and Arrouye.

5. Each Limitation of Claim 21 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

119. Independent method claim 21 is materially identical, for purposes of my analysis, to independent claim 1. A side-by-side chart showing these claims is provided below (with common language underlined):

Claim 1	Claim 21
1. A method of modelling a communications network using a computer system, the method including:	21. A computer program product for modelling a communications network, the computer program product including a computer usable storage medium having computer readable code embodied in the computer usable storage medium, the computer readable code including instructions to:
generating a network representation using computer-readable code, the computer-readable code representing structured information;	generate a network representation, the network representation representing structured information;
parsing the network representation;	parse the network representation; and
generating a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects;	generate a network model using the parsed network representation, the network model including a plurality of network objects and relationships between the plurality of network objects; and
storing the network model in memory; and	
processing a network event using the network model, wherein the processing includes identifying one or more network objects of the plurality of network objects, and the processing further includes determining an order of operation on the one or more network objects.	process a network event using the network model, wherein processing the network event includes identifying one or more network objects of the plurality of network objects, and further includes determining an order of operation on the one or more of the plurality of network objects.

120. As the table above shows, the only material differences between claim 1 and claim 21 is that (1) claim 1 is a method claim and claim 10 is a

computer program product claim, and (2) claim 10 omits the step of "storing the network model in memory" recited in claim 1.

121. These distinctions are immaterial to my analysis. The prior art discloses computer readable medium that include instructions (e.g. software) for performing their operations. (*See, e.g.*, Hamner, 2:56-62 ("The present invention includes steps for providing a number of core services for managing a computer network, such as a local area network (LAN). The steps may be performed by a general purpose processor programmed with instructions that cause the processor to perform the recited steps. The instructions may be stored in a memory or on a magnetic disk, CD-ROM, or any other mass storage device."); Matheny, ¶ 0010 ("The network manager **104** may be a host computer that includes software for initiating and coordinating network discovery operations on devices in the network using a number of different agents."); Arrouye, 4:64-67 ("An embodiment of the invention can be implemented as computer software in the form of computer readable code executed on a general purpose computer such as computer **100** illustrated in FIG. 1…") (underlining added to all).) For the same reasons as claim 1, therefore, claim 21 is disclosed by the Matheny in view of *XML in a Nutshell*, Hamner and Arrouye.

6. Each Limitation of Claim 22 is Disclosed by Matheny, *XML in a Nutshell*, Hamner and Arrouye

122. Dependent method claim 22 is materially identical, for purposes of my analysis, to dependent claim 8. A side-by-side chart showing these claims is provided below (with common language underlined):

Claim 8	Claim 22
8. A method according to claim 1, further including:	22. A computer program product according to claim 21, the computer readable code further including instructions to
<u>generating a graphical representation of the network model</u>; and	<u>generate a graphical representation of the network model</u>; and
<u>displaying the graphical representation on a display monitor</u>.	<u>display the graphical representation on a display monitor</u>.

123. As the table above shows, the only material difference between claim 8 and claim 22 is that the former is a method claim while the latter recites a "computer program product" containing the same steps. As I explained for claim 21 (from which claim 22 depends), the prior art discloses the "computer program product" as claimed. For the same reasons as claims 8 and 21, therefore, claim 22 is disclosed by Matheny in view of *XML in a Nutshell*, Hamner and Arrouye.

B. Claims 1, 7, 8, 10, 21 and 22 Are Obvious Over Matheny in view of *XML in a Nutshell*, Hamner and Pitt

124. In my opinion, the limitations of claims 1, 7, 8, 10, 21 and 22 are disclosed by a second ground: the combination of Matheny, *XML in a Nutshell*, Hamner and Pitt. This second ground is identical to the one I described above in **Part VI.A**, except that I have applied **Pitt** instead of **Arrouye** to the last limitation in independent claims 1, 10 and 21: "determining an order of operation on the one or more network objects." My discussion will therefore focus on how Pitt discloses the "determining an order of operation" limitation of claims 1, 10 and 21, and the rationale for combining Pitt with Matheny, *XML in a Nutshell* and Hamner. My earlier application of the non-Pitt references, *i.e.* Matheny, XML in a Nutshell and Hamner, remains unchanged and applies here.

Overview of Pitt

125. In any networked computer system, certain types of operations must be performed in a particular order or sequence. One of the clearest examples occurs when a portion of a network must be shut down or temporarily removed from the network. If the shutdown or removal of devices from the network is not performed in a specific sequence, data may be lost or corrupted. As Pitt explains:

> When shutting down a computer network, it is important that no portion or segment of the network be in a condition of instability at the time of shutdown, so as not to destroy or corrupt or otherwise deleteriously affect any data in the network, wherever it may be located. This requirement exists for all computer networks, whether a local area network (LAN), a wide area network (WAN), a network encompassing an entire organization (enterprise), or a collection of networks (internet).

(Pitt, 1:29-37.) Pitt further explains:

> Accordingly, upon shutdown and to avoid such deleterious changes in updating data or in transmission of data to or from any computer in the network, <u>specific operations must occur in the following sequence</u>. First, all work stations must be shut down. Second, the database server, prior to its shutdown, must complete that portion of the update to transmit the updated data to the file server. Third, the file server must store the data; only then can it be shut down.

(Pitt, 1:66-2:6 (underlining added).) Following such an ordered sequence, according to Pitt, will help prevent loss of data and ensure its availability when

the network is brought back online. (Pitt, 2:6-10.)

126. Pitt discloses a system for establishing a series of rules to specify an order of operation for individual devices or an entire computer network in response to a shutdown event. (Pitt, 2:16-21.) The shutdown event can occur in response to manual user action, power failure, or pursuant to a timed scheduled. (Pitt, 4:10-12.) The ordering rules are provided to devices on the network in order to perform shutdown operations in the required sequence. (Pitt, 4:66-5:4.)

Pitt Discloses Determining an Order of Operation on Network Objects

127. Pitt discloses the step of processing a network event that includes "determining an order of operation on the one or more network objects," as recited in claim 1. Pitt discloses the ability to create a "shutdown plan" that, when approved by a system administrator, is communicated to the devices on the network. (Pitt, 4:66-5:4.) The ordering rules are used in the event an individual device, or an entire network, is shut down. "Shutdown commences, as denoted by enclosure **58** [Figure 3] by such events or circumstances as by manual activation, by a power failure or by a schedule." (Pitt, 5:11-13.) Regardless of the event that resulted in the shutdown, the rules require that the affected devices shut down at the time and in the sequence specified by the shutdown plan. (Pitt,

5:43-56.) "Because all computers are so shut down in the preprogrammed sequence," Pitt explains, "when the network is later powered up, all data will then be found to have remained in a predictable manner and can be used with confidence." (Pitt, 5:59-63 (underlining added).) Pitt therefore discloses the step of "determining an order of operation on the one or more network objects."

Rationale for Combining Pitt with Matheny and Hamner

128. It would have been obvious to one of ordinary skill in the art to apply the teachings of Pitt to Matheny and Hamner, with no change in the respective functions. This would have predictably resulted in the network management systems of Matheny and Hamner with the additional ability to store information allowing them to ascertain the order of operation for a network device, for example, in response to shutdown event.

129. Pitt confirms that it was well-known to persons of ordinary skill in the art that when shutting down or disabling an individual device or a computer network, certain operations must be performed in a particular order to avoid loss of data. (Pitt, 1:66-2:13.) One of ordinary skill in the art would have understood that determining an order of operation for the network devices, as disclosed in

Pitt, would have improved Matheny and Hamner by allowing them to process network events in a way that avoided loss of data or created other instabilities. These and other advantages were expressly recognized in Pitt:

> Several advantages are derived from this arrangement. A sequential order of shutdown is established. Predictability in shutdown is assured. Deleterious changes in updating the database or in transmission of data to or from the work station are avoided during shutdown, whether through power failure or manual shutdown or timed schedules when the system is to be turned off.

(Pitt, 3:17-23.) One of ordinary skill in the art would therefore have been motivated to adapt the order of operation teachings of Pitt to the network management systems of Matheny and Hamner to achieve these advantages.

130. One of ordinary skill in the art would have regarded Pitt as a natural combination with Matheny and Hamner for several reasons. To begin with, Hamner specifically discloses a database that stores information about tasks that can be performed on the network devices. (Hamner, 10:66-11:7, 11:20-21 ("In the preferred embodiment, available tasks are stored in the physical model database **333**.").) The task database in Hamner could easily have been adapted to

incorporate order of operation information, such as the sequenced shutdown rules disclosed in Pitt. In fact, one of the tasks that can be performed in Hamner is "rebooting selected workstations" (Hamner, 3:54-55), an operation that will generally cause the temporary unavailability of those devices from the network.

131. One of ordinary skill in the art, and almost anyone who has used a modern computer, would have understood that the "reboot" operation mentioned in Hamner involves shutting down all applications and devices running on the computer (including network devices), followed by a restart of the computer. A reboot of a computer, in many ways, is functionally equivalent to removing power from the device, and then turning the device back on. The shutdown plan in Pitt would therefore have provided a benefit to the devices managed by the system of Hamner. For all of these reasons, all limitations of claims 1, 7, 8, 10, 21 and 22 are disclosed by the combination of Matheny, *XML in a Nutshell*, Hamner and Pitt.

VII. CONCLUSION

132. In my opinion, each element of claims 1, 7, 8, 10, 21 and 22 is disclosed or suggested by the prior art references described above, and a person of ordinary skill in the art would have had ample motivation to combine those references. Each challenged claim, therefore, would have been obvious to a person of ordinary skill in the art.

133. In signing this Declaration, I recognize that the Declaration will be filed as evidence in a contested case before the Patent Trial and Appeal Board of the United States Patent and Trademark Office. I also recognize that I may be subject to cross-examination in this proceeding. If required, I will appear for cross-examination at the appropriate time. I reserve the right to offer opinions relevant to the invalidity of the ’300 patent claims at issue and/or offer testimony in support of this Declaration.

134. I hereby declare that all statements made herein of my own knowledge are true and that all statements made on information and belief are believed to be true, and further that these statements were made with the knowledge that willful false statements and the like so made are punishable by fine or imprisonment, or both, under 28 U.S.C. § 1001.

Dated: January 28, 2015 Respectfully submitted,

Tal Lavian, Ph.D.

EXHIBIT A

Tal Lavian, Ph.D.

http://telecommnet.com
http://cs.berkeley.edu/~tlavian
tlavian@telecommnet.com

1640 Mariani Dr.
Sunnyvale, CA 94087
(408)-209-9112

Research and Consulting: Network Communications, Telecommunications, and Internet Protocols

- Scientist, educator, and technologist with over 25 years of experience
- Co-author of over 25 scientific publications, journal articles, and peer-reviewed papers
- Named inventor on over 80 issued and filed patents
- Industry Fellow and Lecturer at UC Berkeley Engineering –Center for Entrepreneurship and Technology (CET)

EDUCATION

- **Ph.D.**, Computer Science specializing in networking and communications, UC Berkeley
- **M.Sc.**, Electrical Engineering, Tel Aviv University
- **B.Sc.**, Mathematics and Computer Science, Tel Aviv University

EXPERTISE

Network communications, telecommunications, mobile wireless and Internet protocols:

- **Communication networks:** Internet Protocols; TCP/IP suite; TCP; UDP; IP; VoIP; Ethernet; network protocols; network software applications; Data Link, Network, and Transport Layers (L2, L3, L4)
- **Routing/switching:** LAN; WAN; VPN; routing protocols; RIP; BGP; MPLS;OSPF; IS-IS;DNS; QoS; switching; packet switching; network infrastructure; network communication architectures
- **Mobile Wireless:** Wireless LAN; 802.11; cellular systems; mobile devices; smartphone technologies
- **Internet Software:** Internet software applications; Internet protocols; distributed computing; Web applications; FTP; HTTP; Java; C; C++; client server; file transfer; multicast; streaming media

LITIGATION SUPPORT SERVICES

- Expert witness in Federal courts, USPTO, and the ITC (over 30 cases)
- Expert witness in USPTO reexamination, interference, and numerous PTAB - IPR
- Expert reports, depositions, and courtroom testimonies
- Skilled articulation of technical material for both technical and non-technical audiences
- Product and technology analysis, patent portfolios, claim charts, patentability research
- Litigation support and technology education in patent disputes, patent reexaminations
- Past cases involved Google, Cisco, Juniper, HP, Ericsson, Microsoft, Apple, and Samsung

ACCOMPLISHMENTS

- Selected as Principal Investigator for three US Department of Defense (DARPA) projects
- Led research project on networking computation for the US Air Force Research Lab (AFRL)
- Led and developed the first network resource scheduling service for grid computing
- Led wireless research project for an undisclosed US federal agency
- Managed and engineered the first demonstrated transatlantic dynamic allocation of 10Gbs Lambdas as a grid service
- Spearheaded and planned the first demonstrated wire-speed active network on commercial hardware
- Invented over 80 patents; over 50 prosecuted *pro se* in front of the USPTO
- Created and chaired Nortel Networks' EDN Patent Committee
- IEEE Senior Member

PROFESSIONAL EXPERIENCE

VisuMenu, Inc. – Sunnyvale, CA 2010-Present

Co- Founder and Chief Technology Officer (CTO)

- Design and develop architecture of visual IVR technologies for smartphones and wireless mobile devices in the area of network communications
- Design crawler/spider system for IVR / PBX using Asterisk, SIP and VoIP
- Deploy the system as cloud networking and cloud computing utilizing Amazon Web Services (EC2, S3, VPC, DNS, and RDS)

Telecomm Net Consulting, Inc. (Innovations-IP) Sunnyvale, CA 2006-Present

Principal Scientist

- Consult in the areas of network communications, telecommunications, Internet protocols, and smartphone mobile wireless devices
- Provide architecture and system consultation for software projects relating to computer networks, mobile wireless devices, Internet web technologies.
- Expert witness in network communications patent infringement suits

University of California Berkeley, Berkeley, CA 2000-Present

Berkeley Industry Fellow, Lecturer, Visiting Scientist, Ph.D. Candidate, Nortel's Scientist Liaison

Some positions and projects were concurrent, others sequential

- Serve as Industry Fellow and Lecturer at the Center for Entrepreneurship and Technology (CET). Studied the areas of network services, telecommunication systems and software, communications infrastructure, and data centers

- Developed long-term technology for the enterprise market, integrating communication and computing technologies
- Conducted research projects in data centers (RAD Labs), telecommunication infrastructure (SAHARA), and wireless systems (ICEBERG)
- Acted as scientific liaison between Nortel Research Lab and UC Berkeley, providing tangible value in advanced technologies
- Earned Ph.D. in Computer Science, specializing in communications and networking

Nortel Networks, Santa Clara, CA 1996 - 2007

Originally employed by Bay Networks, which was acquired by Nortel Networks

Principal Scientist, Principal Architect, Principal Engineer, Senior Software Engineer

- Held scientific and research roles at Nortel Labs, Bay Architecture Labs, and CTO Office

Principal Investigator for US Department of Defense (DARPA) Projects

- Conceived, proposed, and completed three research projects: Active Networks, DWDM-RAM, and a networking computation project for Air Force Research Lab (AFRL)
- Led a wireless research project for an undisclosed US federal agency

Academic and Industrial Researcher

- Analyzed new technologies to reduce risks associated with R&D investment
- Spearheaded research collaboration with leading universities and professors at UC Berkeley, Northwestern University, University of Amsterdam, and University of Technology Sydney
- Evaluated competitive products relative to Nortel's products and technology
- Proactively identified prospective business ideas, leading to new networking products
- Predicted technological trends in advance through researching the technological horizon and academic sphere
- Developed software for switches, routers and network communications devices
- Developed systems and architectures for switches, routers, and network management
- Researched and developed the following projects:
  - Data-Center Communications: network and server orchestration 2006-2007
  - DRAC: SOA-facilitated L1/L2/L3 network dynamic controller 2003-2007
  - Omega: classified wireless project for undisclosed US Federal Agency 2006
  - Open Platform: project for the US Air Force Research Laboratory (AFRL) 2005
  - Network Resource Orchestration for Web Services Workflows 2004-2005

- Proxy Study between Web/Grids Services and Network Services 2004
- Streaming Content Replication: real-time A/V media multicast at edge 2003-2004
- DWDM-RAM: US DARPA-funded program on agile optical transport 2003-2004
- Packet Capturing and Forwarding Service on IP and Ethernet traffic 2002-2003
- CO2: content-aware agile networking 2001-2003
- Active Networks: US DARPA-funded research program 1999-2002
- ORE: programmable network service platform 1998-2002
- JVM Platform: Java on network devices 1998-2001
- Web-Based Device Management: network device management 1996-1997

Technology Innovator and Patent Leader

- Created and chaired Nortel Networks' EDN Patent Committee
- Facilitated continuous stream of innovative ideas and their conversion into intellectual property rights
- Developed intellectual property assets through invention and analysis of existing technology portfolios

Aptel Communications, Netanya, Israel 1994-1995

Software Engineer, Team Leader

Start-up company focused on mobile wireless CDMA spread spectrum PCN/PCS

- Developed mobile wireless device using an unlicensed band, Direct Sequence Spread Spectrum (DSSS)
- Designed and managed a personal communication network (PCN) and personal communication system (PCS), the precursors of short text messages (SMS)
- Responsible for the design and development of network software products
- Developed software network communications mainly in C/C++
- Brought two-way paging product from concept to development

Scitex Ltd.,Herzeliya, Israel 1990-1993

Software Engineer, Team Leader

Software and hardware company acquired by Hewlett Packard (HP)

- Developed system and network communications mainly in C/C++
- Invented Parallel SIMD Architecture
- Participated in the Technology Innovation group

Shalev, Ramat-HaSharon, Israel 1987-1990

Start-up company

Software Engineer

- Developed real-time software and algorithms mainly in C/C++ and Pascal

PROFESSIONAL ASSOCIATIONS

- IEEE Senior Member
- IEEE CNSV co-chair Intellectual Property SIG (2013)
- President Next Step Toastmasters (an advanced TM club in the Silicon Valley) (2013)
- Technical Co-Chair, IEEE Hot Interconnects 2005 at Stanford University
- Member, IEEE Communications Society (COMMSOC)
- Member, IEEE Computer Society
- Member, IEEE Systems, Man, and Cybernetics Society
- Member, IEEE-USA Intellectual Property Committee
- Member, ACM, ACM Special Interest Group on Data Communication (SIGCOM)
- Member, ACM Special Interest Group on Hypertext, Hypermedia and Web (SIGWEB)
- Member, IEEE Consultants' Network (CNSV)
- Global Member, Internet Society (ISOC)
- President Java Users Group – Silicon Valley Mountain View, CA,1999-2000
- Toastmasters International

ADVISORY BOARDS

- Quixey – (present) search engine for wireless mobile apps
- Mytopia – mobile social games
- iLeverage – Israeli Innovations

PROFESSIONAL AWARDS

- Top Talent Award – Nortel
- Top Inventors Award – Nortel EDN
- Certified IEEE-WCET - Wireless Communications Engineering Technologies
- Toastmasters International - Competent Communicator (twice)
- Toastmasters International - Advanced Communicator Bronze

Patents and Publications

(notan exhaustive list)

Patents Issued:

- **US 8,688,796** Rating system for determining whether to accept or reject objection raised by user in social network
- **US 8,572,303** Portable universal communication device
- **US 8,553,859** Device and method for providing enhanced telephony
- **US 8,548,131** Systems and methods for communicating with an interactive voice response system
- **US 8,537,989** Device and method for providing enhanced telephony
- **US 8,341,257** Grid proxy architecture for network resources
- **US8,161,139** Method and apparatus for intelligent management of a network element
- **US 8,146,090** Time-value curves to provide dynamic QoS for time sensitive file transfer
- **US 8,078,708** Grid proxy architecture for network resources
- **US 7,944,827** Content-aware dynamic network resource allocation
- **US7,860,999** Distributed computation in network devices
- **US 7,734,748** Method and apparatus for intelligent management of a network element
- **US 7,710,871** Dynamic assignment of traffic classes to a priority queue in a packet forwarding device
- **US 7,580,349** Content-aware dynamic network resource allocation
- **US 7,433,941** Method and apparatus for accessing network information on a network device
- **US 7,359,993** Method and apparatus for interfacing external resources with a network element
- **US 7,313,608** Method and apparatus for using documents written in a markup language to access and configure network elements
- **US 7,260,621** Object-oriented network management interface

- **US 7,237,012** Method and apparatus for classifying Java remote method invocation transport traffic
- **US 7,127,526** Method and apparatus for dynamically loading and managing software services on a network device
- **US7,047,536** Method and apparatus for classifying remote procedure call transport traffic
- **US7,039,724** Programmable command-line interface API for managing operation of a network device
- **US6,976,054** Method and system for accessing low-level resources in a network device
- **US6,970,943** Routing architecture including a compute plane configured for high-speed processing of packets to provide application layer support
- **US6,950,932** Security association mediator for Java-enabled devices
- **US6,850,989** Method and apparatus for automatically configuring a network switch
- **US6,845,397** Interface method and system for accessing inner layers of a network protocol
- **US6,842,781** Download and processing of a network management application on a network device
- **US6,772,205** Executing applications on a target network device using a proxy network device
- **US6,564,325** Method of and apparatus for providing multi-level security access to system
- **US6,175,868** Method and apparatus for automatically configuring a network switch
- **US6,170,015** Network apparatus with Java co-processor
- **US 8,619,793** Dynamic assignment of traffic classes to a priority queue in a packet forwarding device
- **US 8687,777** Systems and methods for visual presentation and selection of IVR menu
- **US 8,681,951** Systems and methods for visual presentation and selection of IVR menu

- **US 8,625,756** Systems and methods for visual presentation and selection of IVR menu
- **US 8,594,280** Systems and methods for visual presentation and selection of IVR menu
- **US 8,548,135** Systems and methods for visual presentation and selection of IVR menu
- **US 8,406,388** Systems and methods for visual presentation and selection of IVR menu
- **US 8,345,835** Systems and methods for visual presentation and selection of IVR menu
- **US 8,223,931** Systems and methods for visual presentation and selection of IVR menu
- **US 8,160,215** Systems and methods for visual presentation and selection of IVR menu
- **US 8,155,280** Systems and methods for visual presentation and selection of IVR menu
- **US 8,054,952** Systems and methods for visual presentation and selection of IVR menu
- **US 8,000,454** Systems and methods for visual presentation and selection of IVR menu
- **EP 1,905,211** Technique for authenticating network users
- **EP 1,142,213** Dynamic assignment of traffic classes to a priority queue in a packet forwarding device
- **EP 1,671,460** Method and apparatus for scheduling resources on a switched underlay network
- **CA 2,358,525** Dynamic assignment of traffic classes to a priority queue in a packet forwarding device

Patent Applications Published and Pending:

- **US 20140105025** Dynamic Assignment of Traffic Classes to a Priority Queue in a Packet Forwarding Device
- **US 20140105012** Dynamic Assignment of Traffic Classes to a Priority Queue in a Packet Forwarding Device
- **US 20140012991** Grid Proxy Architecture for Network Resources
- **US 20130080898** Systems and Methods for Electronic Communications
- **US 20130022191** Systems and Methods for Visual Presentation and Selection of IVR Menu
- **US 20130022183** Systems and Methods for Visual Presentation and Selection of IVR Menu
- **US 20130022181** Systems and Methods for Visual Presentation and Selection of IVR Menu
- **US 20120180059** Time-Value Curves to Provide Dynamic QOS for Time Sensitive File Transfers
- **US 20120063574** Systems and Methods for Visual Presentation and Selection of IVR Menu
- **US 20110225330** Portable Universal Communication Device
- **US 20100220616** Optimizing Network Connections
- **US 20100217854** Method and Apparatus for Intelligent Management of a Network Element
- **US 20100146492** Translation of Programming Code
- **US 20100146112** Efficient Communication Techniques
- **US 20100146111** Efficient Communication in a Network
- **US 20090313613** Methods and Apparatus for Automatic Translation of a Computer Program Language Code
- **US 20090313004** Platform-Independent Application Development Framework
- **US 20090279562** Content-aware dynamic network resource allocation
- **US 20080040630** Time-Value Curves to Provide Dynamic QoS for Time Sensitive File Transfers
- **US 20070169171** Technique for authenticating network users
- **US 20060123481** Method and apparatus for network immunization
- **US 20060075042** Extensible Resource Messaging Between User Applications and Network Elements in a Communication Network

- **US 20050083960** Method and Apparatus for Transporting Parcels of Data Using Network Elements with Network Element Storage
- **US 20050076339** Method and Apparatus for Automated Negotiation for Resources on a Switched Underlay Network
- **US 20050076336** Method and Apparatus for Scheduling Resources on a Switched Underlay Network
- **US 20050076173** Method And Apparatus for Preconditioning Data to Be Transferred on a Switched Underlay Network
- **US 20050076099** Method and Apparatus for Live Streaming Media Replication in a Communication Network
- **US 20050074529** Method and apparatus for transporting visualization information on a switched underlay network
- **US 20040076161** Dynamic Assignment of Traffic Classes to a Priority Queue in a Packet Forwarding Device
- **US 20020021701** Dynamic Assignment of Traffic Classes to a Priority Queue in a Packet Forwarding Device
- **WO 2007/008976** Technique for Authenticating Network Users
- **WO 2006/063052** Method and apparatus for network immunization
- **WO2000/0054460** Method and apparatus for accessing network information on a network device

Publications

(not an exhaustive list)

- "Communications Architecture in Support of Grid Computing",Tal Lavian,Scholar's Press 2013ISBN 978-3-639-51098-0.
- "Applications Drive Secure Lightpath Creation across Heterogeneous Domains, Feature Topic Optical Control Planes for Grid Networks: Opportunities, Challenges and the Vision." Gommans L.; Van Oudenaarde B.; Dijkstra F.; De Laat C.; Lavian T.; Monga I.; Taal A.; Travostino F.; Wan A.; IEEE *Communications Magazine*, vol. 44, no. 3, March 2006, pp. 100-106.
- *Lambda Data Grid: Communications Architecture in Support of Grid Computing*. Tal I. Lavian, Randy H. Katz; Doctoral Thesis, University of California at Berkeley. January 2006.
- "Information Switching Networks." Hoang D.B.; T. Lavian;*The 4th Workshop on the Internet, Telecommunications and Signal Processing, WITSP2005*, December 19-21, 2005, Sunshine Coast, Australia.
- "Impact of Grid Computing on Network Operators and HW Vendors." Allcock B.; Arnaud B.; Lavian T.; Papadopoulos P.B.; Hasan M.Z.; Kaplow W.; *IEEE Hot Interconnects at Stanford University 2005*, pp.89-90.
- *DWDM-RAM: AData Intensive Grid Service Architecture Enabled by Dynamic Optical Networks*. Lavian T.; Mambretti J.; Cutrell D.; Cohen H.J; Merrill S.; Durairaj R.; Daspit P.; Monga I.; Naiksatam S.; Figueira S.; Gutierrez D.; Hoang D.B., Travostino F.; CCGRID 2004, pp. 762-764.
- *DWDM-RAM: AnArchitecture for Data Intensive Service Enabled by Next Generation Dynamic Optical Networks*. Hoang D.B.; Cohen H.; Cutrell D.; Figueira S.; Lavian T.; Mambretti J.; Monga I.; Naiksatam S.; Travostino F.; Proceedings IEEE Globecom 2004, Workshop on High-Performance Global Grid Networks, Houston, 29 Nov. to 3 Dec. 2004, pp.400-409.
- *Implementation of a Quality of Service Feedback Control Loop on Programmable Routers*. Nguyen C.; Hoang D.B.; Zhao, I.L.; Lavian, T.; Proceedings, 12th IEEE International Conference on Networks 2004. (ICON 2004) Singapore, Volume 2, 16-19 Nov. 2004, pp.578-582.
- *A Platform for Large-Scale Grid Data Service on Dynamic High-Performance Networks*. Lavian T.; Hoang D.B.; Mambretti J.; Figueira S.; Naiksatam S.; Kaushil N.; Monga I.; Durairaj R.; Cutrell D.; Merrill S.; Cohen H.; Daspit P.; Travostino F; GridNets 2004, San Jose, CA., October 2004.
- *DWDM-RAM: Enabling Grid Services with Dynamic Optical Networks*. Figueira S.; Naiksatam S.; Cohen H.; Cutrell D.; Daspit, P.; Gutierrez D.; Hoang D. B.; Lavian T.; Mambretti J.; Merrill S.; Travostino F; Proceedings, 4th IEEE/ACM International Symposium on Cluster Computing and the Grid, Chicago, USA, April 2004, pp. 707-714.
- *DWDM-RAM: Enabling Grid Services with Dynamic Optical Networks*. Figueira S.; Naiksatam S.; Cohen H.; Cutrell D.; Gutierrez D.; Hoang D.B.; Lavian T.; Mambretti J.; Merrill S.; Travostino F.; 4th IEEE/ACM International Symposium on Cluster Computing and the Grid, Chicago, USA, April 2004.
- *An Extensible, Programmable, Commercial-Grade Platform for Internet Service Architecture*. Lavian T.; Hoang D.B.; Travostino F.; Wang P.Y.; Subramanian S.; Monga I.; IEEE Transactions on Systems, Man, and

Cybernetics on Technologies Promoting Computational Intelligence, Openness and Programmability in Networks and Internet Services Volume 34, Issue 1, Feb. 2004, pp.58-68.

- *DWDM-RAM: An Architecture for Data Intensive Service Enabled by Next Generation Dynamic Optical Networks*. Lavian T.; Cutrell D.; Mambretti J.; Weinberger J.; Gutierrez D.; Naiksatam S.; Figueira S.; Hoang D. B.; Supercomputing Conference, SC2003 Igniting Innovation, Phoenix, November 2003.
- *Edge Device Multi-Unicasting for Video Streaming*. Lavian T.; Wang P.; Durairaj R.; Hoang D.; Travostino F.; Telecommunications, 2003. ICT 2003. 10th International Conference on Telecommunications, Tahiti, Volume 2, 23 Feb.-1 March, 2003 pp. 1441-1447.
- The SAHARA Model for Service Composition Across Multiple Providers. Raman B.; Agarwal S.; Chen Y.; Caesar M.; Cui W.; Lai K.; Lavian T.; Machiraju S.; Mao Z. M.; Porter G.; Roscoe T.; Subramanian L.; Suzuki T.; Zhuang S.; Joseph A. D.; Katz Y.H.; Stoica I.; Proceedings of the First International Conference on Pervasive Computing. ACM Pervasive 2002, pp. 1-14.
- *Enabling Active Flow Manipulation in Silicon-Based Network Forwarding Engines*. Lavian T.; Wang P.; Travostino F.; Subramanian S.; Duraraj R.; Hoang D.B.; Sethaput V.; Culler D.; Proceeding of the Active Networks Conference and Exposition, 2002.(DANCE) 29-30 May 2002, pp. 65-76.
- *Practical Active Network Services within Content-Aware Gateways*. Subramanian S.; Wang P.; Durairaj R.; Rasimas J.; Travostino F.; Lavian T.; Hoang D.B.; Proceeding of the DARPA Active Networks Conference and Exposition, 2002.(DANCE) 29-30 May 2002, pp. 344-354.
- *Active Networking on a Programmable Network Platform.* Wang P.Y.; Lavian T.; Duncan R.; Jaeger R.; Fourth IEEE Conference on Open Architectures and Network Programming (OPENARCH), Anchorage, April 2002.
- *Intelligent Network Services through Active Flow Manipulation*. Lavian T.; Wang P.; Travostino F.; Subramanian S.; Hoang D.B.; Sethaput V.; IEEE Intelligent Networks 2001 Workshop (IN2001), Boston, May 2001.
- Intelligent Network Services through Active Flow Manipulation. Lavian T.; Wang P.; Travostino F.; Subramanian S.; Hoang D.B.; Sethaput V.; Intelligent Network Workshop, 2001 IEEE 6-9 May 2001, pp.73 - 82.
- *Enabling Active Flow Manipulation in Silicon-based Network Forwarding Engine*. Lavian, T.; Wang, P.; Travostino, F.; Subramanian S.; Hoang D.B.; Sethaput V.; Culler D.; Journal of Communications and Networks, March 2001, pp.78-87.
- *Active Networking on a Programmable Networking Platform*. Lavian T.; Wang P.Y.; IEEE Open Architectures and Network Programming, 2001, pp. 95-103.
- *Enabling Active Networks Services on a Gigabit Routing Switch*. Wang P.; Jaeger R.; Duncan R.; Lavian T.; Travostino F.; 2nd Workshop on Active Middleware Services, 2000.
- *Dynamic Classification in Silicon-Based Forwarding Engine Environments*. Jaeger R.; Duncan R.; Travostino F.; Lavian T.; Hollingsworth J.; Selected Papers. 10th IEEE Workshop on Metropolitan Area and Local Networks, 1999. 21-24 Nov. 1999, pp.103-109.

- *Open Programmable Architecture for Java-Enabled Network Devices*. Lavian, T.; Jaeger, R. F.; Hollingsworth, J. K.; IEEE Hot Interconnects Stanford University, August 1999, pp. 265-277.
- *Open Java SNMP MIB API*. Rob Duncan, Tal Lavian, Roy Lee, Jason Zhou, Bay Architecture Lab Technical Report TR98-038, December 1998.
- *Java-Based Open Service Interface Architecture*. Lavian T.; Lau S.; BAL TR98-010 Bay Architecture Lab Technical Report, March 1998.
- *Parallel SIMD Architecture for Color Image Processing*. Lavian T. Tel – Aviv University, Tel – Aviv, Israel, November 1995.
- *Grid Network Services, Draft-ggf-ghpn-netservices-1.0***.**George Clapp, Tiziana Ferrari, Doan B. Hoang, Gigi Karmous-Edwards, Tal Lavian, Mark J. Leese, Paul Mealor, Inder Monga, Volker Sander, Franco Travostino, Global Grid Forum(GGF).
- *Project DRAC: Creating an applications-aware network.*Travostino F.; Keates R.; Lavian T.; Monga I.; Schofield B.; Nortel Technical Journal, February 2005, pp. 23-26.
- *Optical Network Infrastructure for Grid, Draft-ggf-ghpn-opticalnets-1.*Dimitra Simeonidou, Reza Nejabati, Bill St. Arnaud, Micah Beck, Peter Clarke, Doan B. Hoang, David Hutchison, Gigi Karmous-Edwards, Tal Lavian, Jason Leigh, Joe Mambretti, Volker Sander, John Strand, Franco Travostino, Global Grid Forum(GGF) GHPN Standard GFD-I.036 August 2004.
- *Popeye - Using Fine-grained Network Access Control to Support Mobile Users and Protect Intranet Hosts.*Mike Chen, Barbara Hohlt, Tal Lavian, December 2000.

Presentations and Talks

(not an exhaustive list)

- Lambda Data Grid: An Agile Optical Platform for Grid Computing and Data-intensive Applications.
- Web Services and OGSA
- WINER Workflow Integrated Network Resource Orchestration.
- Technology & Society.
- Abundant Bandwidth and how it affects us?
- Active Content Networking(ACN).
- DWDM-RAM:Enabling Grid Services with Dynamic Optical Networks .
- Application-engaged Dynamic Orchestration of Optical Network Resources .
- A Platform for Data Intensive Services Enabled by Next Generation Dynamic Optical Networks .
- Optical Networks.
- Grid Optical Network Service Architecture for Data Intensive Applications.
- Optical Networking & DWDM.
- OptiCal Inc.
- OptiCal & LUMOS Networks.
- Optical Networking Services.
- Business Models for Dynamically Provisioned Optical Networks.
- Business Model Concepts for Dynamically Provisioned Optical Networks.
- Optical Networks Infrastructure.
- Research Challenges in agile optical networks.
- Services and Applications' infrastructure for agile optical networks.
- Impact on Society.
- TeraGrid Communication and Computation.
- Unified Device Management via Java-enabled Network Devices.
- Active Network Node in Silicon-Based L3 Gigabit Routing Switch.
- Active Nets Technology Transfer through High-Performance Network Devices.
- Programmable Network Node: Applications.
- Open Innovation via Java-enabled Network Devices.
- Practical Considerations for Deploying a Java Active Networking Platform.
- Open Java-Based Intelligent Agent Architecture for Adaptive Networking Devices.
- Java SNMP Oplet.
- Open Distributed Networking Intelligence: A New Java Paradigm.
- Open Programmability.
- Active Networking On A Programmable Networking Platform.
- Open Networking through Programmability.
- Open Programmable Architecture for Java-enabled Network Devices.

- Integrating Active Networking and Commercial-Grade Routing Platforms.
- Programmable Network Devices.
- To be smart or not to be?